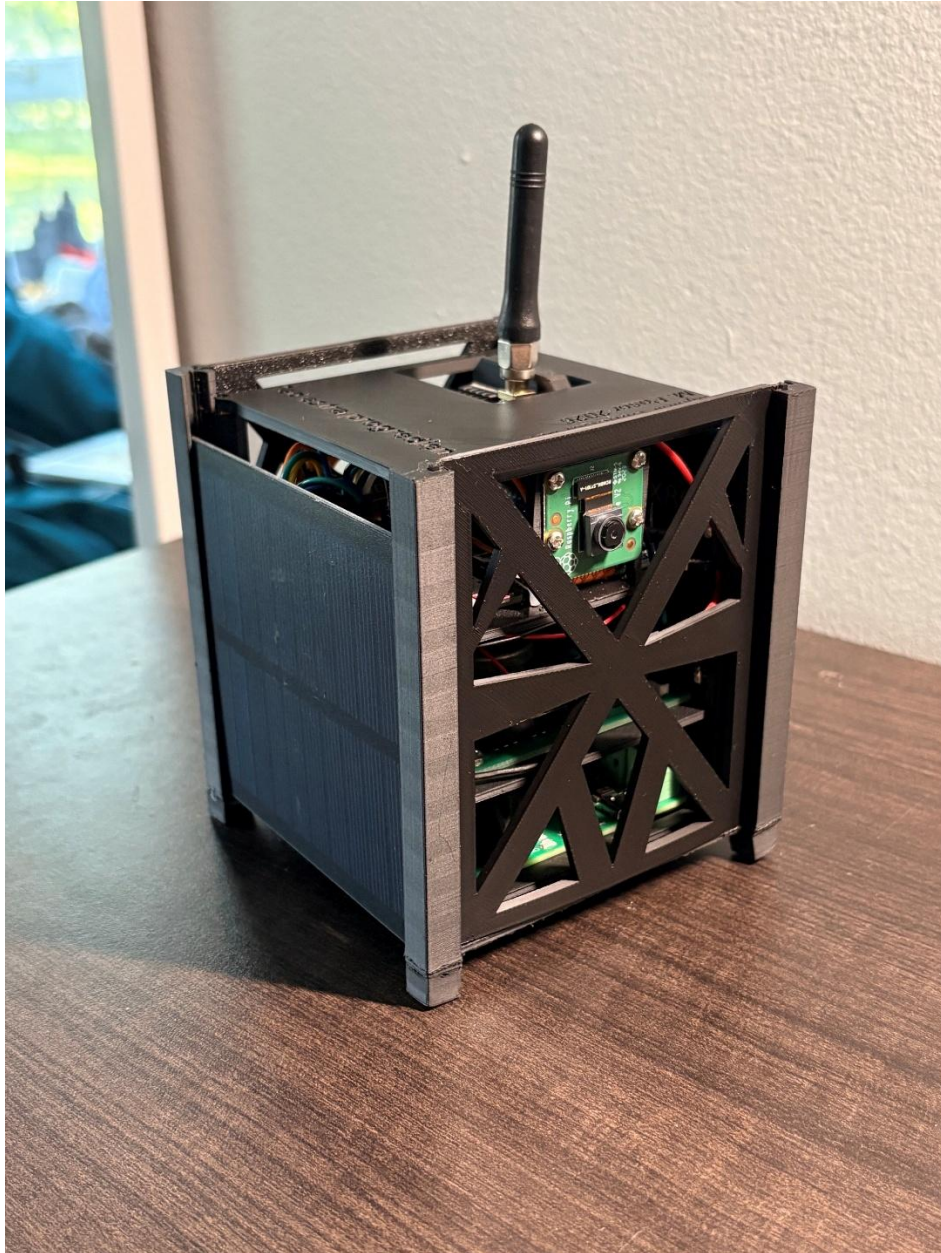


Mjolnir CubeSat Design & System Architecture Report



Matt Poser, Queen's University Engineering Physics

September 17th, 2026

Preamble/Abstract

The goal of this document is to notarize the design decisions, architecture, and detailed subsystems of the Mjolnir CubeSat personal project (similarly to my earlier M.A.E.S.T.R.O document) now that the Mk.I version is complete. This project was initially conceived when doing job market research and looking at the RocketLab Toronto engineering team, responsible for constructing reaction wheels and star tracker components for sale to the general space industry. Upon further research, I found out about student space projects, and specifically the CubeSat design standard used by industry engineers, hobbyists, and university students across the world. I set out to learn more and make my own, to learn about the relevant design challenges, subsystems and components commonly seen within the space industry and on operational satellites.

From a high level, the Mjolnir (Mk.I) CubeSat is a 1U design made from 3D printed PLA (fast prototyping), complete with an onboard Raspberry Pi Zero2W (flight computer), PiCamera V2, Adafruit LoRa radio module (915MHz), Adafruit BNO055 9 degree-of-freedom inertial measurement board, a custom made reaction wheel motor control PCB & reaction wheels, and a fully custom 5V output Lithium Ion (18650 cells) battery management system with a custom designed PCB. The CubeSat is specifically designed to communicate with a 'ground station' comprised of my laptop (running frontend display & backend software) paired with an Arduino Uno & accompanying Adafruit LoRa module which can receive onboard sensor telemetry, as well as send commands to change onboard configurations on the flight computer.

In its current state, the Mk.I is a simple prototype (and notably **NOT** spaceflight worthy), meant to act as a learning tool about PCB design, 3D modeling, software design, and relevant space applications. It works fully as intended, broadcasting down sensor telemetry packets and accepting commands from the ground station during active operation. Being a 1U design, there was not much room for scientific equipment or significant 'launch payloads', however if I ever were to attempt a 2U (or larger) footprint CubeSat I would attempt this, and possibly make it launch-worthy. I had an absolute blast with this project, and I am hoping to use it as a talking point in interviews and on job applications (if you're a recruiter reading this, get in touch!). I set out on this project to learn, and to improve my skills in 3D modeling, software, and PCB design, and achieved all of those goals in full.

- Matt Poser, Sept. 17th 2026

Table of Contents

Mjolnir CubeSat Design &	1
System Architecture Report	1
Preamble/Abstract	2
Project Motivation, Background	5
Initial LoRa Radio Test	8
General System Architecture, Operation	10
Mechanical Design, Tray System	12
Tray I – BMS & Power Supply Board	15
Tray II – Reaction Wheels	20
Tray III – Flight Computer, IMU, Radio, Startracker	24
Flight Computer Software.....	27
Groundstation, Software & Hardware.....	29
Final Assembly, Future Project Considerations.....	30

Figure 1- Star Trackers, from the RocketLab Website: https://rocketlabcorp.com/space-systems/satellite-components/star-trackers/	5
Figure 2- Reaction wheels, from the RocketLab website: https://rocketlabcorp.com/space-systems/satellite-components/reaction-wheels/	6
Figure 3- 1U CubeSat footprint design standard, CubeSat official foundation	7
Figure 4- Adafruit LoRa modules, selected to be used both onboard the CubeSat and on the groundstation	8
Figure 5- Soldering job of the two purchased Adafruit radio modules. Note the safety goggles on the left, PPE is important!.....	9
Figure 6- Photo of the 'test bench' setup for the LoRa radio hardware test. See the 'groundstation' Arduino on the left hand side, and the Pi 4B on the right hand side	9
Figure 7- Note the shell output from the Pi side of the ping pong script. Messages were actively being exchanged back and forth between the two radios, on regular intervals, for sustained periods of time	10
Figure 8- Initial CubeSat tower design. Struts and flat base follow 1U design standard	12
Figure 9- Model for CubeSat frame with risers and machined holes for heat set inserts	13
Figure 10- Heat set inserts installed in the Mk.I outer frame	13
Figure 11- CubeSat outer frame with teal tray template prints installed at various heights within the CubeSat. These were used to confirm design dimension, as well as verify fit and the ideas behind the tiered system	14
Figure 12- used a Dremel tool to file down the slots of the CubeSat in an attempt to fit the solar panels. The height dimension was correct, however in the end the frame required a redesign of the slots in order to accommodate the solar panels as wall units	14

Figure 13- CubeSat Lid, the last component to be designed for the structural system and the CubeSat as a whole. Latin saying roughly translates to "forward unto the stars"	15
Figure 14- Power input circuit, with diode OR circuit to feed charging chip	16
Figure 15- Power path charging chip circuit	17
Figure 16- Battery protection circuit	17
Figure 17- Power output circuit	18
Figure 18- BMS design & routing. Note the square footprint to fit the tiered tray design	18
Figure 19- KiCAD render of BMS board	19
Figure 20- BMS board soldered and assembled from JLCPCB.....	19
Figure 21- BMS Power test	20
Figure 22- 5V DC motors off amazon	21
Figure 23- Motor control PCB circuit, using two of the Texas Instruments L293DNE ICs. Two dual H bridges meant 4 total motors could be controlled, but I only required 3 axes of control, thus one was left empty	21
Figure 24- motor control traces.....	22
Figure 25- KiCAD 3D render of the motor control PCB	23
Figure 26- Motor control PCB soldered, and fastened to reaction wheel tray along with reaction wheel motors. Flywheels would be printed and assembled later, along with all wiring connections	23
Figure 27- basic layout of Adafruit BNO055 IMU board, Mjolnir flight computer, LoRa radio module	24
Figure 28- top tray assembled on frame with BMS.....	25
Figure 29- Top tray assembled on frame with BMS (2)	25
Figure 30- Top tray with PiCamera mounting arms extruded from the base of the tray	26
Figure 31- top tray assembled with PiCamera and LoRa radio module	26
Figure 32- IMU sub-script, interfaces with Adafruit IMU to obtain attitude and temperature readings, publishes sensor packet to an mqtt topic read by the main LoRa script, which is then sent down to the groundstation as a sensor packet.....	28
Figure 33- Mjolnir CubeSat groundstation operator panel, designed fully in JavaScript	29
Figure 34- Arduino Uno groundstation module. Yellow serial cable plugs into the USB port of my laptop, to interface with python backend	30
Figure 35- Initial electronics test. See the mess of wires coming out of the flight computer, the 3D printed flywheels on the reaction wheel motor controller, and the bench power supply being used to conserve battery (in lieu of).....	31
Figure 36- Photos of various stages of final CubeSat assembly.....	35
Figure 37- Mjolnir CubeSat, Mk.I	36

Project Motivation, Background

As I mentioned in the executive summary portion of this document, I had initially been looking into jobs for various internship roles while at Queen's & on my internship at OPG. I had found a posting for RocketLab's engineering team which worked on making reaction wheels and star trackers for satellite applications, and at the time I had yet to learn what either of these were. Intrigued, I did some research and found documentation both star trackers and reaction wheels, as well as their use in spacecraft design. For the un-initiated, a star tracker is a special kind of camera which takes photos of the stars while a spacecraft is in orbit or traveling through space. The star tracker then maps this photo to a known star map (360° panoramic view of the night sky), using specially designed software to determine the vector direction that the camera (and thus spacecraft) is pointing in, as well as the angular orientation. This is an important task for navigation of spacecraft, as well as calibrating the onboard orientation sensors to a known source.



Figure 1- Star Trackers, from the RocketLab Website: <https://rocketlabcorp.com/space-systems/satellite-components/star-trackers/>

The next component I had yet to learn about was reaction wheels. Once your spacecraft has its exact angular orientation in space determined (combination of star tracker & onboard sensors), the operators back on earth (or onboard) may wish to change the angular orientation. This could be to face solar panels towards the sun, change attitude (angular orientation) to face a celestial body/source for imaging, or other reasons. You could accomplish this with thrusters, however reaction wheels also achieve the task of changing angular orientation in space, via the conservation of angular momentum. Let's frame the problem as the following. Your spacecraft has a total moment of inertia $I_{spacecraft}$. If we define the total angular momentum of the spacecraft as $L_{spacecraft}$, we can represent it with the following equation:

$$L_{spacecraft} = I_{spacecraft} * \omega \quad (1)$$

Where ω is the angular momentum of the entire spacecraft body, with all of its components. We will say, that buy and large after launch, our spacecraft is rotating a minimal amount in a nearby observer's frame of reference. Thus, $L_{spacecraft}$ is equal to 0. Since we know that $I_{spacecraft}$ is

nonzero (our spacecraft has mass), then we know that our spacecraft's total angular momentum (ω) must be 0. Thus, our equation (1) becomes:

$$0 = I_{spacecraft} * \omega$$

Now, what we will do is break up the above equation into a combination of simple parts. We can say that the total angular momentum of our spacecraft, is equal to the sum of the angular momentum of smaller components within the spacecraft, along with the spacecraft's outer body. For now, we will formalize this along one arbitrary axis, saying we have a single internal component which can take angular momentum ω_2 . Our equation (1) then becomes:

$$0 = I_{body} * \omega_1 + I_{flywheel} * \omega_2 \quad (2)$$

We can see, that for the above equation to remain valid, either $\omega_2 = \omega_1 = 0$, or they must take opposite signs and cancel one another out, as we specify that both I_{body} and $I_{flywheel}$ are positive real tensors (though, in this single axis approximation we can think of them as scalar values). What this means is that if there is no external addition of angular momentum to the entire spacecraft assembly, you can still induce angular momentum in the external spacecraft body by making an internal component spin (have nonzero angular momentum). This in turn allows the crew to rotate the spacecraft, altering its attitude in a known and controlled way. This internal rotating component used to change spacecraft attitude, is the reaction wheel. Perhaps an easier (and less long winded) way to think about this is to say that if our spacecraft starts at rest (no angular momentum), and a wheel inside starts spinning in one direction, the whole spacecraft (external body) must rotate in the other direction to conserve angular momentum, which remains zero for the entire spacecraft.

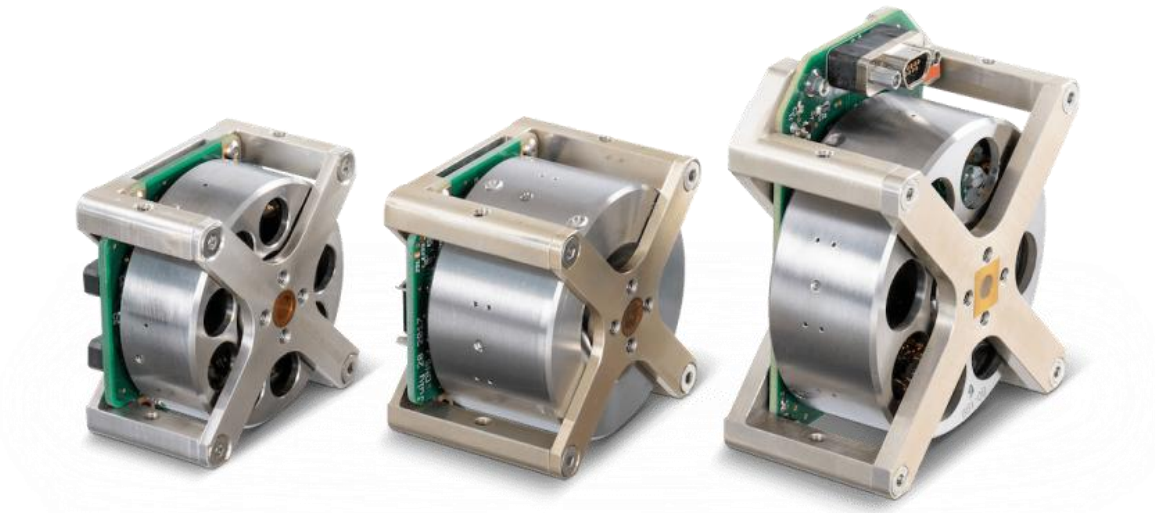


Figure 2- Reaction wheels, from the RocketLab website: <https://rocketlabcorp.com/space-systems/satellite-components/reaction-wheels/>

Having learned about all of this, I was now curious about the industry of satellite design. I also found several resources pointing to CubeSats; used for both real-world space deployments and by university teams. CubeSats (as I have come to know) are satellites which follow a specific design standard, limiting their total physical footprint to multiples of 10cm/10cm/10cm. A singular 1000cm³ satellite is referred to as a 1U (1 unit) CubeSat, and different multiples of this are referred to aptly as 2U, 3U, so on and so forth up to 12U satellites. Shown below is a photo of the official 1U design standard from the CubeSat foundation:

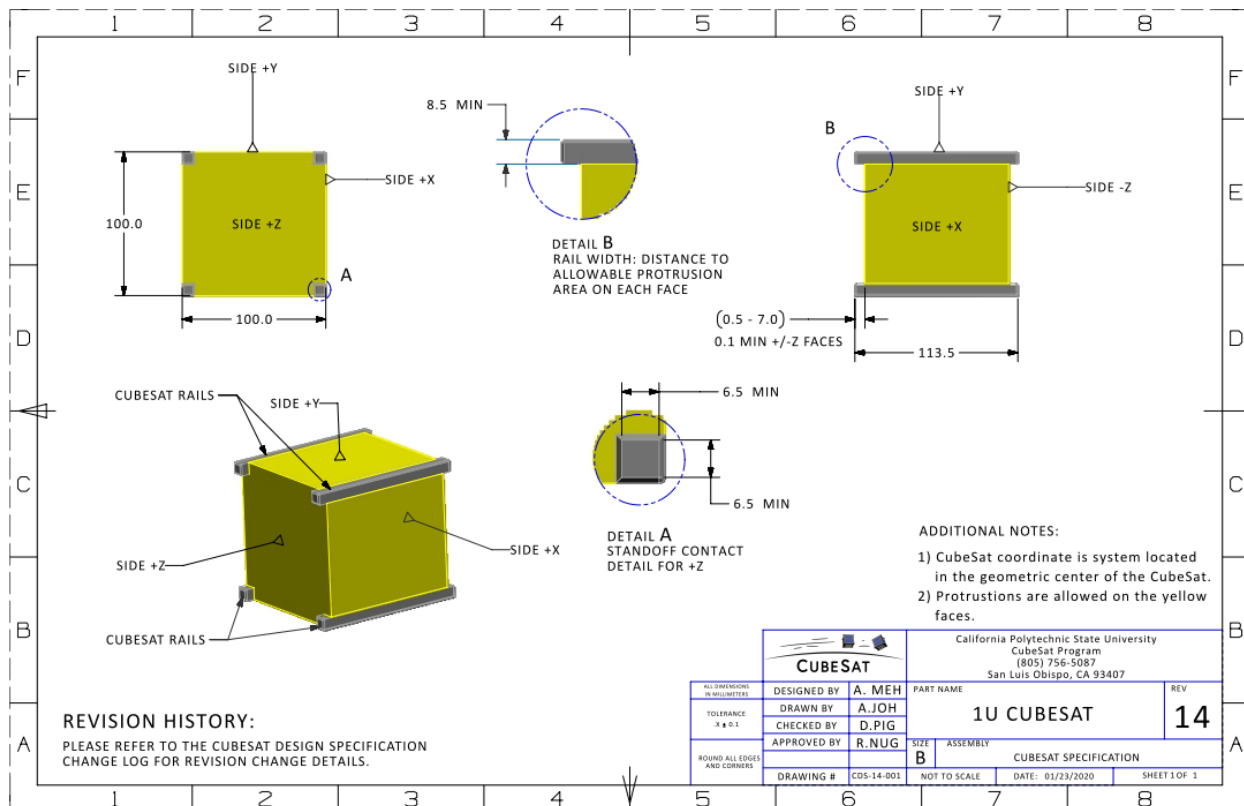


Figure 3- 1U CubeSat footprint design standard, CubeSat official foundation

The best thing about CubeSats for both students and professionals is that they were invented to circumvent the costs of launching satellites into orbit. Before CubeSats, launching a satellite meant that an organization would spend potentially millions of dollars building and launching a behemoth sized instrument on a rocket. This made the practice of satellite design and operation inaccessible at an entry level. With the much smaller footprint CubeSats, engineers and physicists can collaborate to design satellites for scientific purposes on a smaller scale for reduced cost and can afford to launch via rideshare. Instead of a company paying 6 (or more) figures to use a dedicated launch vehicle to put only their satellite into orbit, rideshares via companies like NASA or SpaceX will carry and launch hundreds of these CubeSats at a time on a single launch vehicle, allowing participating parties to collectively pay for the launch vehicle operation & reduce the cost of putting their experiment in space.

By this point I was thoroughly intrigued, and wanted to take on the project of building my own CubeSat (despite not being able to afford the roughly \$50,000 - \$80,000 rideshare launch bill). I

began project planning for a 1U CubeSat of my own, and made a laundry list of technical skills I hoped to practice and improve upon during this project. In late April 2026, I kicked off the project and began documentation.

Initial LoRa Radio Test

Similarly to my M.A.E.S.T.R.O desktop robotic arm project (also on my GitHub, check it out!), this project too relied on a core concept/component, and without proper operation/functionality the project would not be able to be attempted. For this project, that concept was radio communication via an I/O module, in this case a LoRa radio module. LoRa radio simply stands for long range radio, and is a special standard of radio communication which is used to send signals & information from node to node. LoRa radios are considered useful for simple & low power radio applications (with surprisingly good range), and can be configured in networks and meshes for message relaying. In this application, my understanding was that my CubeSat (Mjolnir, Mk.I) would require an onboard radio module which could both send and receive messages/packets. This onboard radio would need to communicate with a groundstation of some kind, built to receive onboard information from the CubeSat and also send commands to the onboard flight computer. I planned outright to use a Raspberry Pi Zero 2W running headless Linux as my flight computer, making use of the GPIO pins and python as a powerful I/O base for onboard sensors & modules. Using this as initial inspiration, I settled on the Adafruit RFM9x 915MHz LoRa modules. These modules were relatively low power and used GPIO based SPI (serial peripheral interface) communication protocol to operate. Further, there was a supported software library for both the python operated GPIO on the raspberry Pi (Adafruit custom), as well as the Arduino microcontroller family (radiohead library). The choice of an Arduino (Uno) is discussed later in the groundstation section of the report, but ultimately the flight computer radio module was to be operated by the onboard Raspberry Pi Zero 2W flight computer & python, while the groundstation would use the Uno & radiohead.

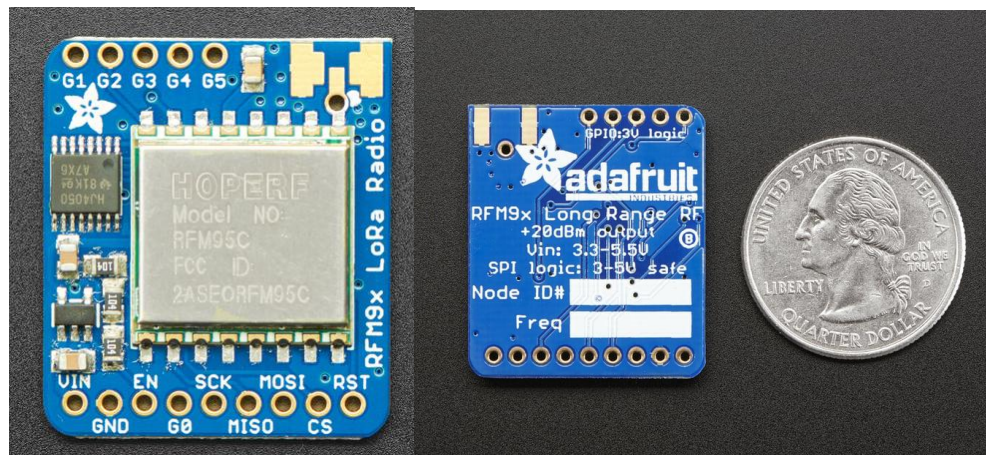


Figure 4- Adafruit LoRa modules, selected to be used both onboard the CubeSat and on the groundstation

When purchased and delivered, the antennae had to be soldered onto both boards, using a special SMD (surface mounted device) for antenna connection. This was my first experience doing non through-hole soldering, and took a lot of patience to complete. Eventually, I was able to finish the job & assemble both radio boards for use:



Figure 5- Soldering job of the two purchased Adafruit radio modules. Note the safety goggles on the left, PPE is important!

Once finished with the soldering job, I wrote up a quick 'ping pong' script for both the Arduino, and a Raspberry Pi 4B (nice to develop on a desktop environment, while still using python/GPIO the same way as the Zero 2W would), where one radio would send out a ping, and wait for the other radio to respond with a 'pong' return message. Once wired up on both the Pi 4B and Arduino Uno, the test ran successfully verifying that these LoRa modules could be used, as well as the fact that my soldering job held up (which, in and of itself, was no small act of divine intervention).

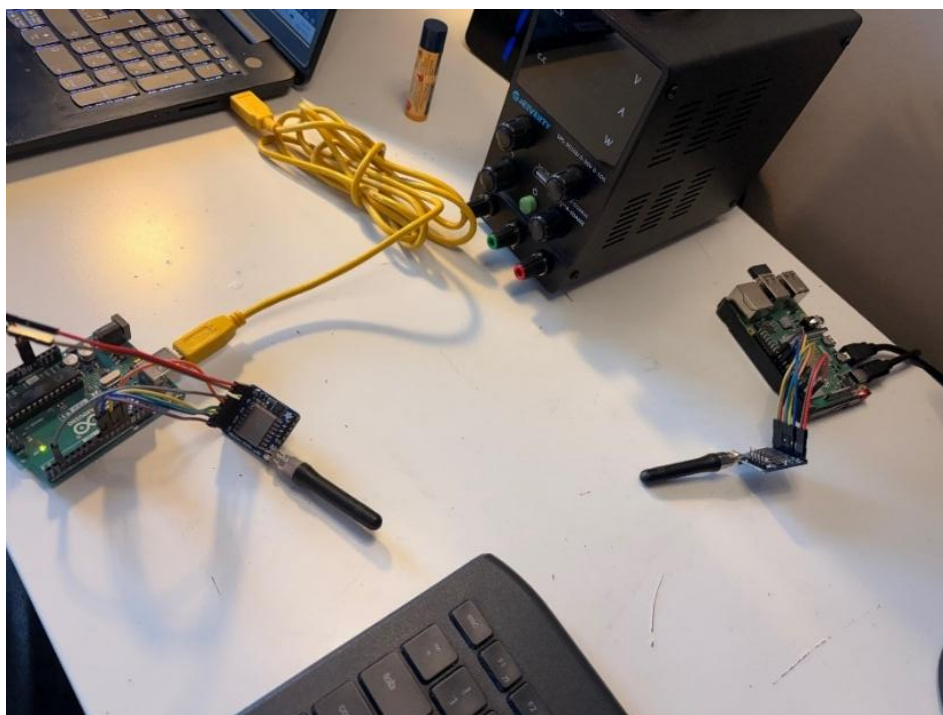


Figure 6- Photo of the 'test bench' setup for the LoRa radio hardware test. See the 'groundstation' Arduino on the left hand side, and the Pi 4B on the right hand side

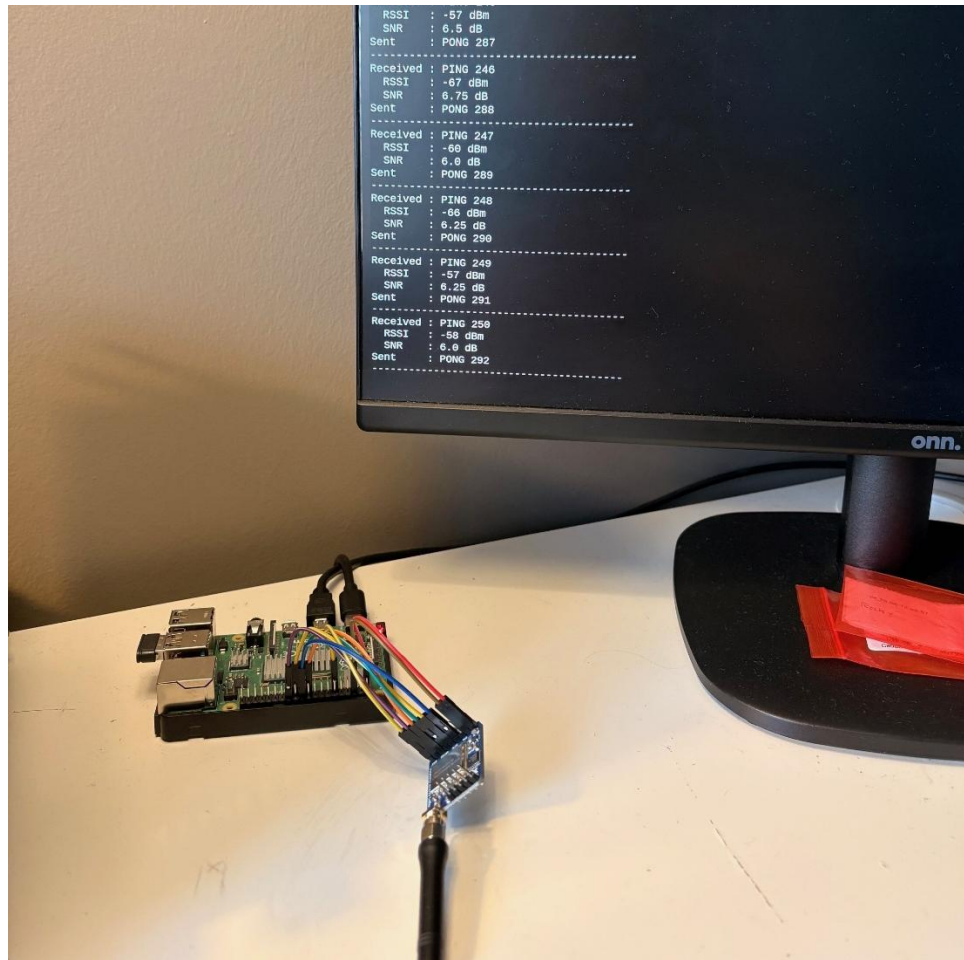


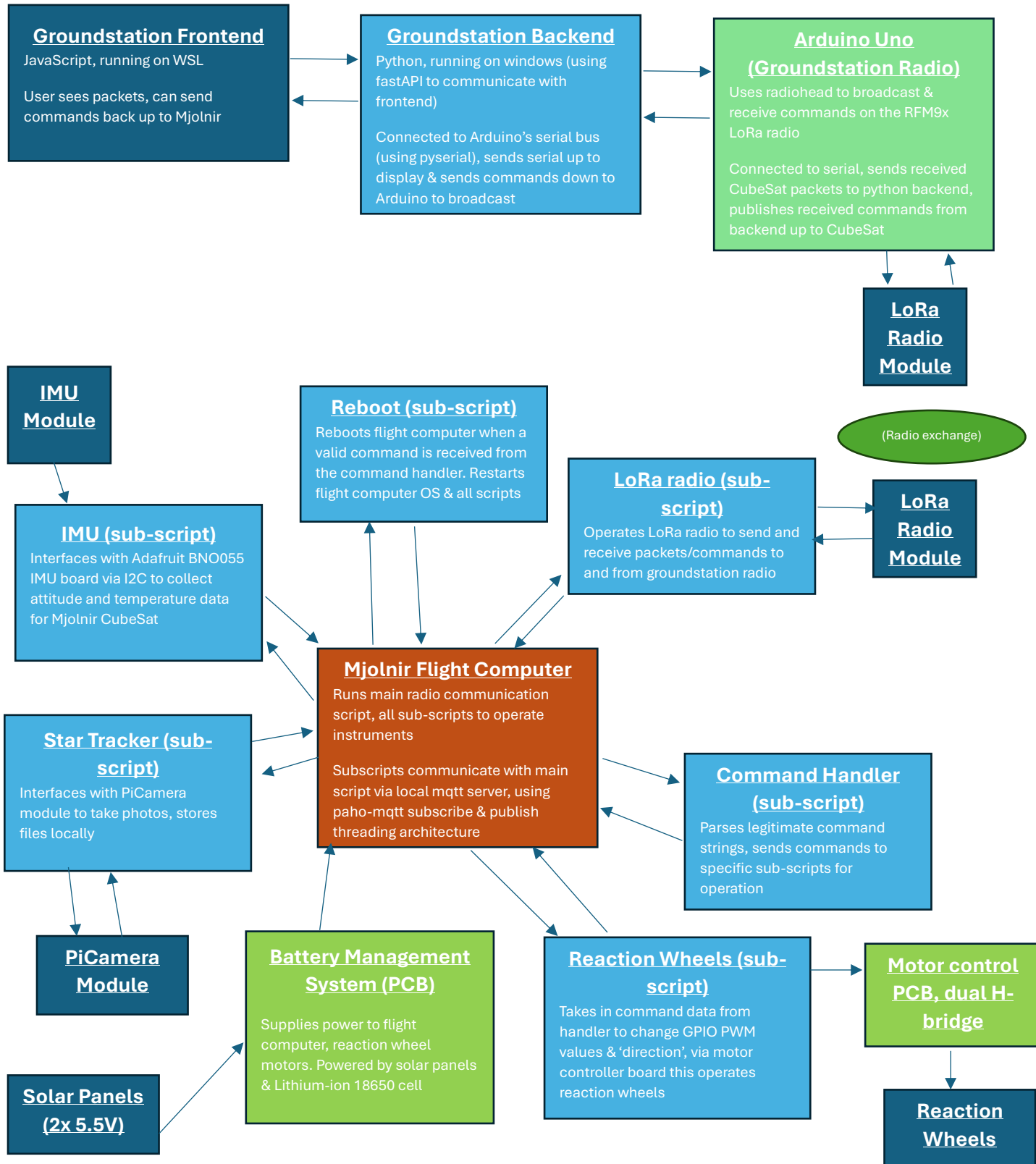
Figure 7- Note the shell output from the Pi side of the ping pong script. Messages were actively being exchanged back and forth between the two radios, on regular intervals, for sustained periods of time

With this successful test, the project could now be fully attempted. If the radio modules did not end up working, then alternative options would have been explored until a viable choice was found for the project. However, this easy use case meant that I could proceed with a general architecture of CubeSat and groundstation in mind.

General System Architecture, Operation

The general architecture of the CubeSat project is that of an uplink/downlink communication scheme. Under regular operation, I wanted the CubeSat to publish regular packets down to the groundstation, alongside other system/operational information as needed. The groundstation would display this information to the operator via a nice frontend display panel, which doubled as an interactive and easy to use control station to send commands to the CubeSat. Onboard the CubeSat, I would need to mesh onboard instruments such as the LoRa radio module, reaction wheels, star tracker camera, and inertial measurement sensor (angular orientation/attitude measurements, temperature) with the Pi Zero2W flight computer. The flight computer would dually

need to send packets down in a known protocol to be processed by the ground station, as well as receive and process known commands coming from the ground station to perform preset flight computer operations. Below is the block diagram format of both the groundstation & CubeSat high level architecture:



Mechanical Design, Tray System

The work for this project began with the design of the physical enclosure to hold all components within the CubeSat. At this point, I had brainstormed general components which would be included in the CubeSat's design, however I did not have a concrete idea as to how I should mount and fasten them when inside the frame. I had the idea that the physical enclosure should follow a 'tower' design, with mounting points along vertical struts for components to be screwed in. I designed the tower to dimension with the 1U standard, and was able to produce the following initial 3D model:

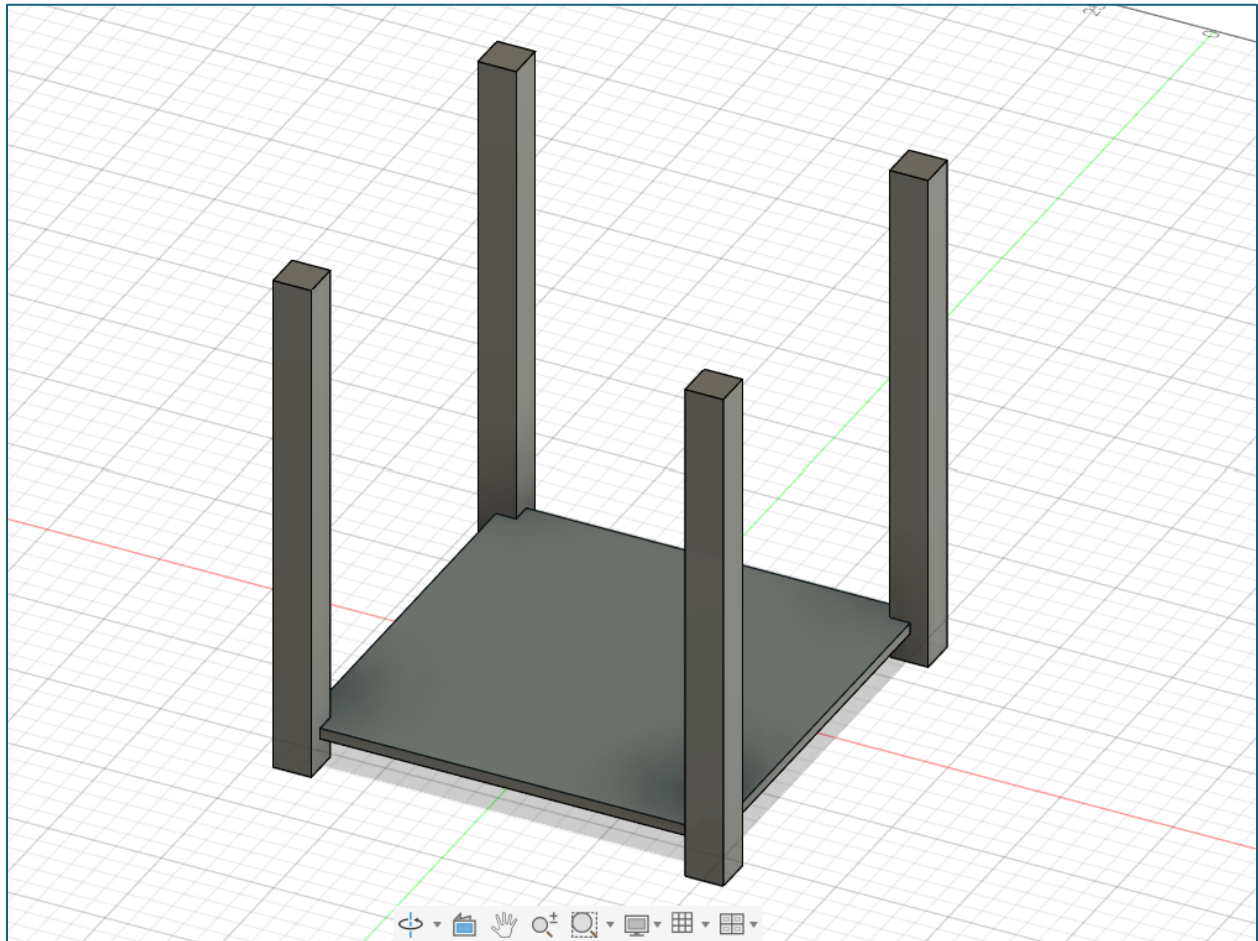


Figure 8- Initial CubeSat tower design. Struts and flat base follow 1U design standard

To mount objects to these towers, I designed in further risers which would hold heat set inserts. Planning on 3D printing these prototypes for this project, heat set inserts allowed me to have a way to use metric fasteners (M3 for this project) to mount all of my components to the frame. Heat set inserts are used by machining a pre-drilled hole in a 3D print, and with a soldering iron pressing in the inserts (heat melts away the plastic around them) to hold them securely in place. This provides a secure threaded anchor for M3 screws. Photos are shown below of the frame's 3D model, and the heat set inserts assembled into the mounting points.

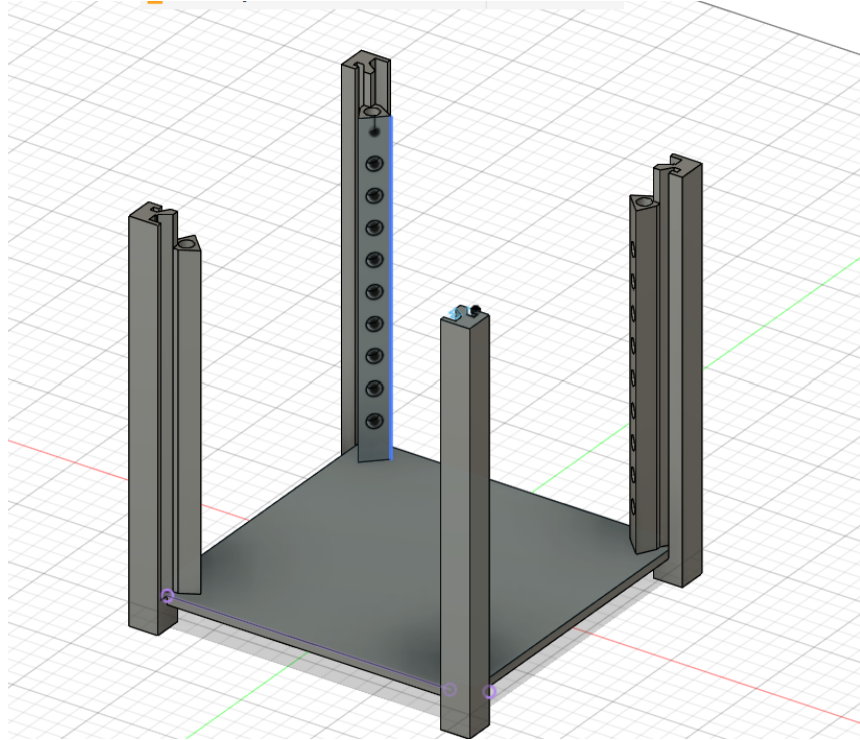


Figure 9- Model for CubeSat frame with risers and machined holes for heat set inserts



Figure 10- Heat set inserts installed in the Mk.I outer frame

In the designing of these towers, I also elected to implement slots for future installations of walls, as at the moment I did not know what they would look like exactly. Zooming in on the top of the tower in figure 10 shows this, there are acceptor slots which would allow a general wall design to be slid in from the top and held in place by a lid.

After the installation of the heat set inserts on the first frame, I had the idea of implementing a tiered 'tray' design. If I had a general footprint for a platform which could be fastened into the heat set inserts via M3 screw, I could start with the same template sketch for all trays which I wanted to install in the CubeSat. This also made it a modular design, meaning in theory trays could be swapped in and out and installed in different configurations. Shown below are some photos taken later in the CubeSat's design phase, including the teal-colored tray template prints fastened to the frame design, as well as the purchased solar panels inserted into machined slots in the frame (solar panels were big enough to act as the CubeSat walls).



Figure 11- CubeSat outer frame with teal tray template prints installed at various heights within the CubeSat. These were used to confirm design dimension, as well as verify fit and the ideas behind the tiered system



Figure 12- used a Dremel tool to file down the slots of the CubeSat in an attempt to fit the solar panels. The height dimension was correct, however in the end the frame required a redesign of the slots in order to accommodate the solar panels as wall units

The structural subsystem was the least intensive part of this project, and was also the first component to undergo design work (the frame), as well as the last (the lid). A photo of the final CubeSat Lid 3D model is shown below, this lid screws into the top of the frame towers to cap off the top tray (containing flight computer, IMU, radio, and camera module).

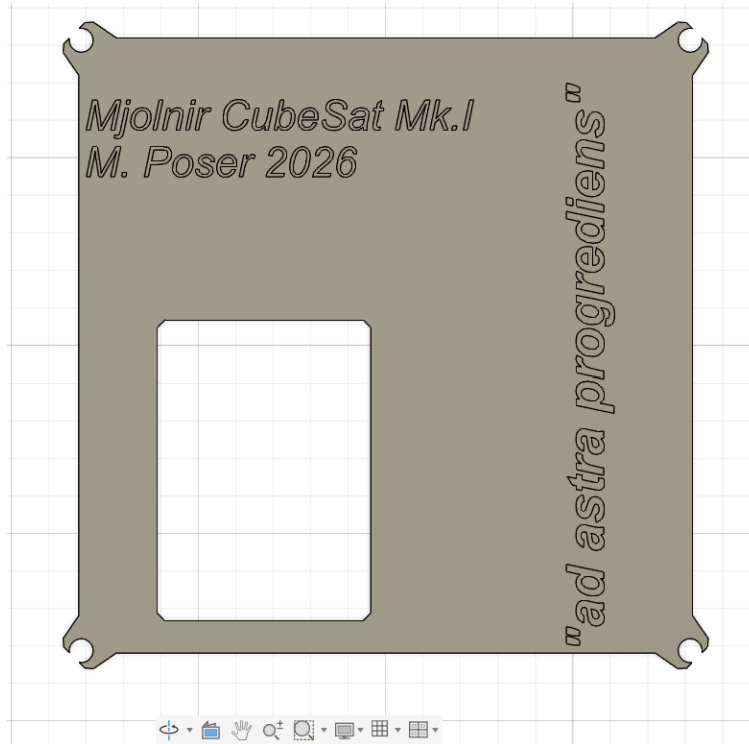


Figure 13- CubeSat Lid, the last component to be designed for the structural system and the CubeSat as a whole. Latin saying roughly translates to "forward unto the stars"

Tray I – BMS & Power Supply Board

The BMS (battery management system) for this CubeSat was an immense amount of work, and presented a very large learning curve for me in terms of PCB design. When talking about this project with one of the engineers on my team at OPG, he convinced me to try and make a custom made BMS, as opposed to purchasing a basic one off the market (my initial inclination). He recommended that I begin with chip design, and lay out a couple of key functionalities for the board:

1. Board must be powered by an 18650 Lithium ion cell, 3.7V nominal
2. Board must have power path management, take in power from both the battery source and the solar panel charging units
3. Board must have aforementioned 18650 cell protection (undervoltage, overvoltage, overcurrent, temperature)
4. Board must take the varying output voltage from the battery/solar panels and boost it up to a consistent 5V

For these basic specifications, I selected the TI-BQ24074 charging chip, TI-BQ29704 battery protection chip, and the TPS61023 boost converter chip for power output. I essentially followed the

exact sample designs from the Texas Instruments design manuals for each chip, stringing each chip together in the schematic to complete the design. I also elected to include a USB-C power input on my BMS PCB, considering that much of the testing I would be conducting would occur at my desk, and I had a high current desk power supply on hand for quick battery charging. Screenshots of each major component/circuit are shown below, with KiCAD comments outlining the specific function of each circuit on the board:

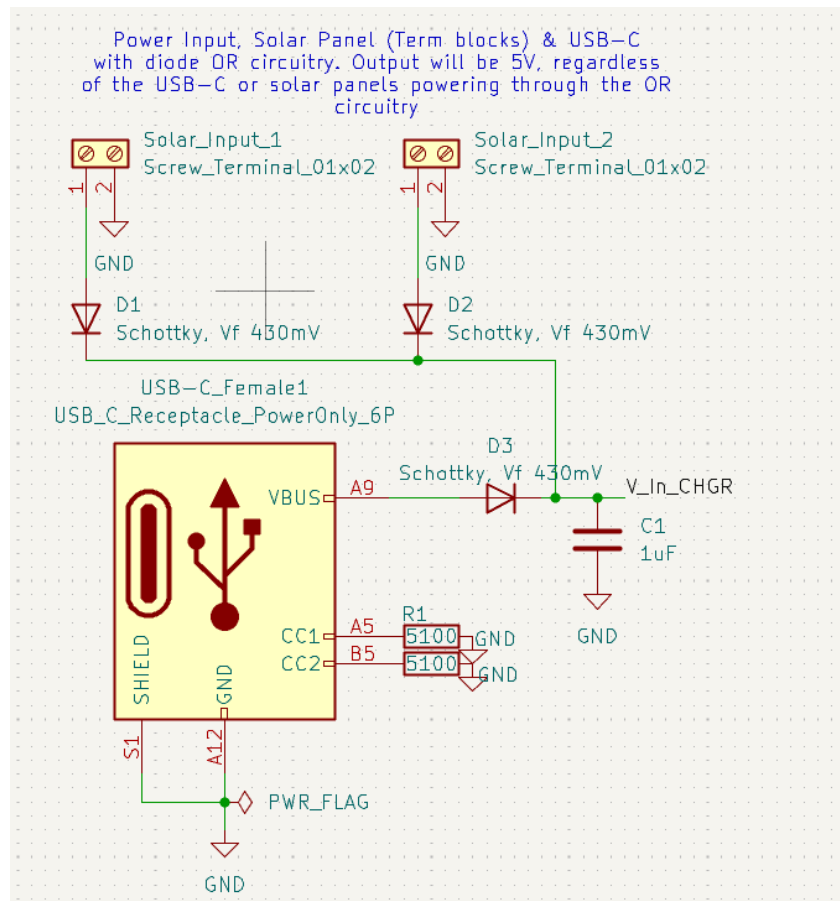


Figure 14- Power input circuit, with diode OR circuit to feed charging chip

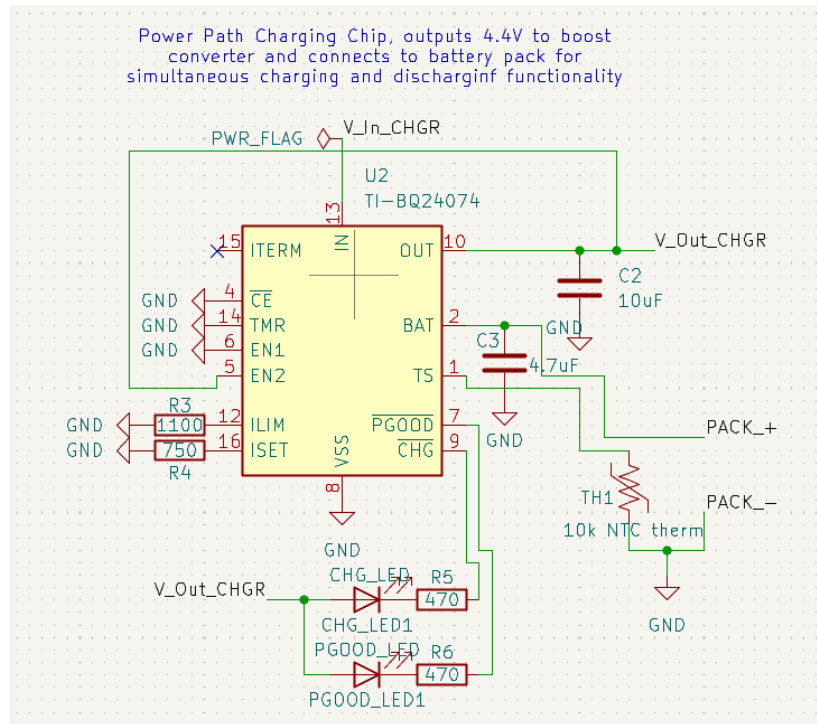


Figure 15- Power path charging chip circuit

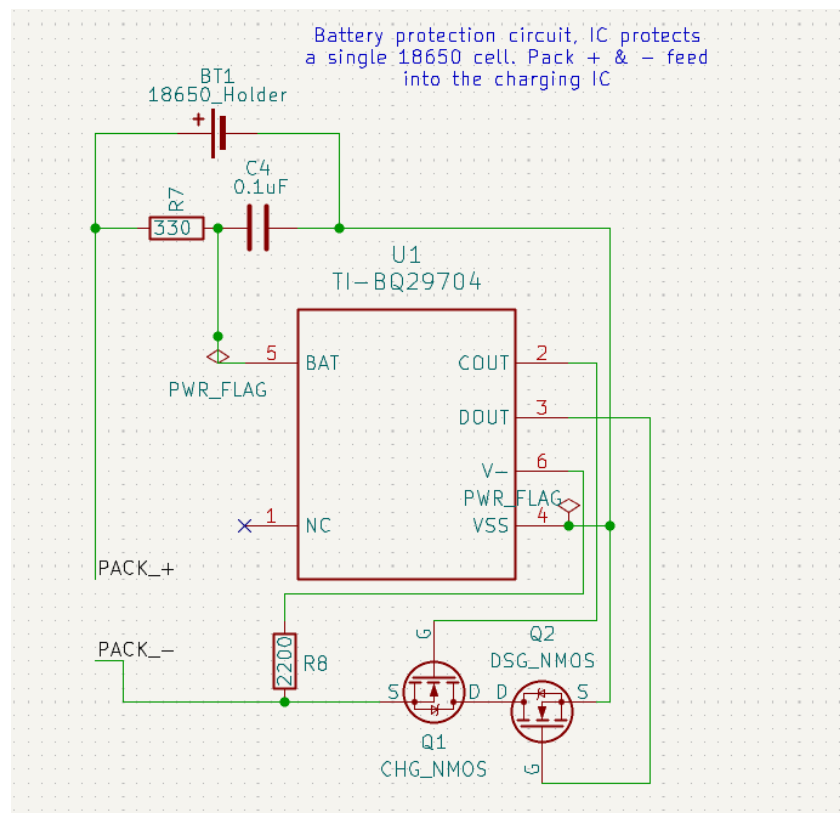


Figure 16- Battery protection circuit

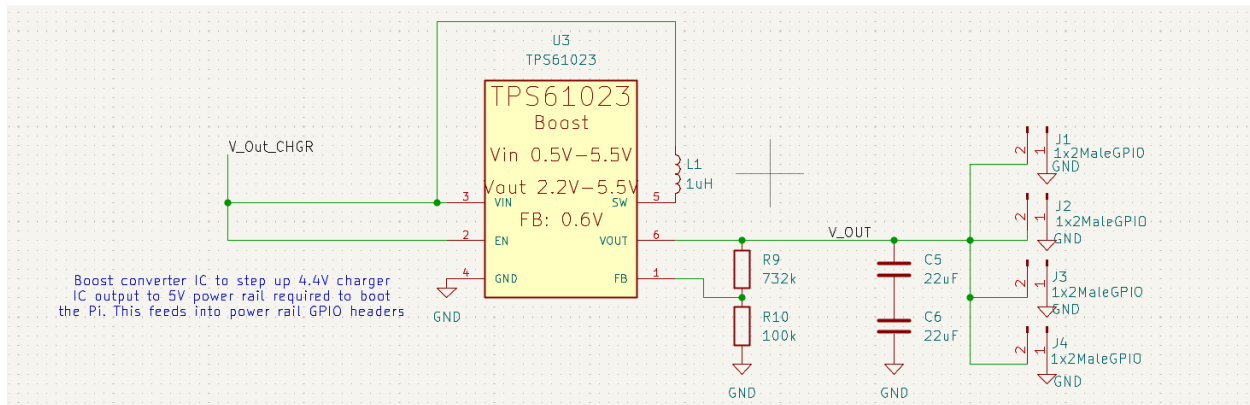


Figure 17- Power output circuit

After much effort routing the design (and a couple revisions after difficulties sourcing parts from the JLCPCB catalog), I was able to create the BMS power supply and place it for order.

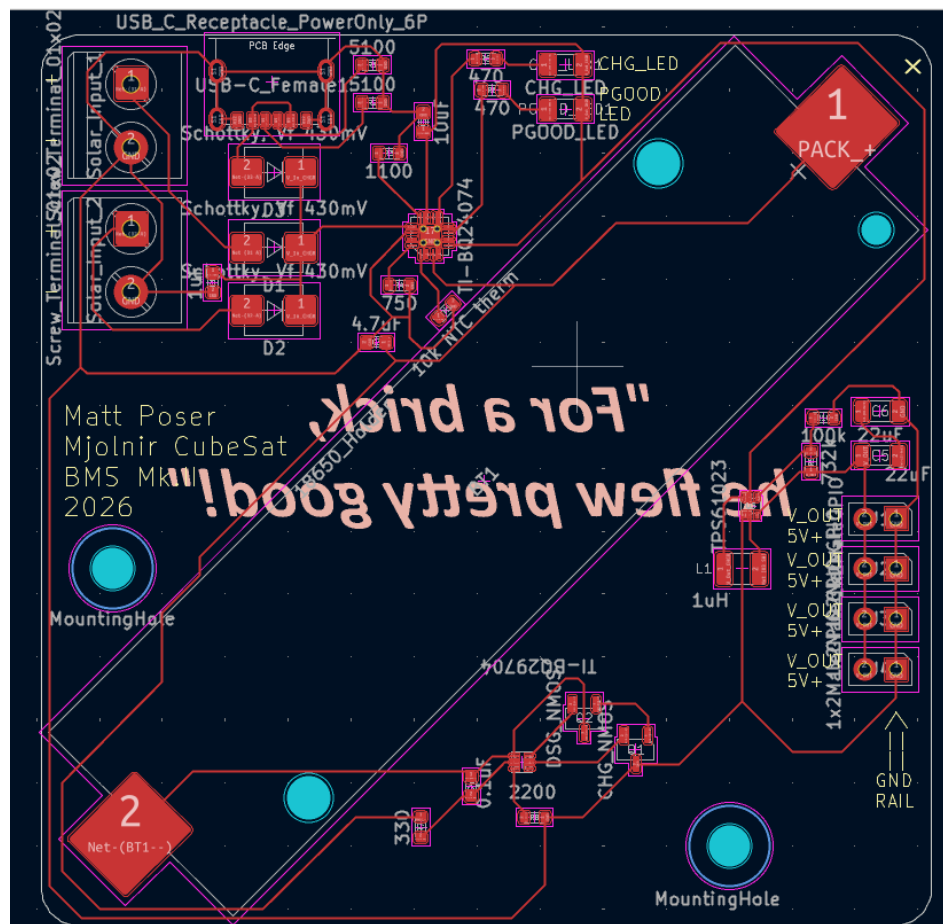


Figure 18- BMS design & routing. Note the square footprint to fit the tiered tray design. Saying on the back comes from Halo 2's Cairo station cutscene

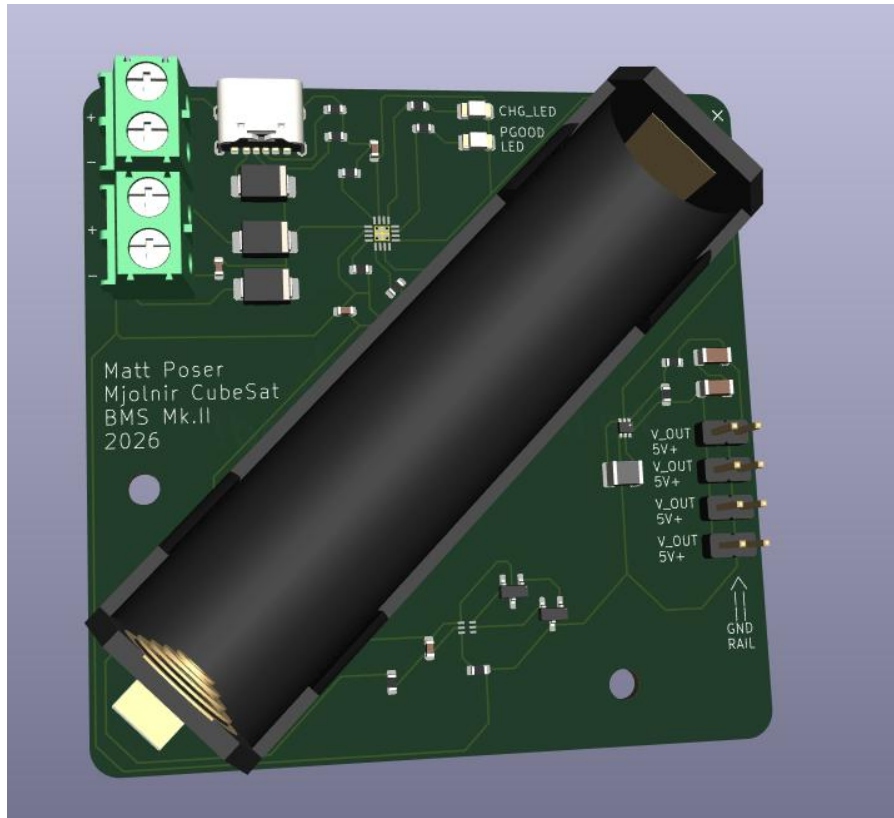


Figure 19- KiCAD render of BMS board

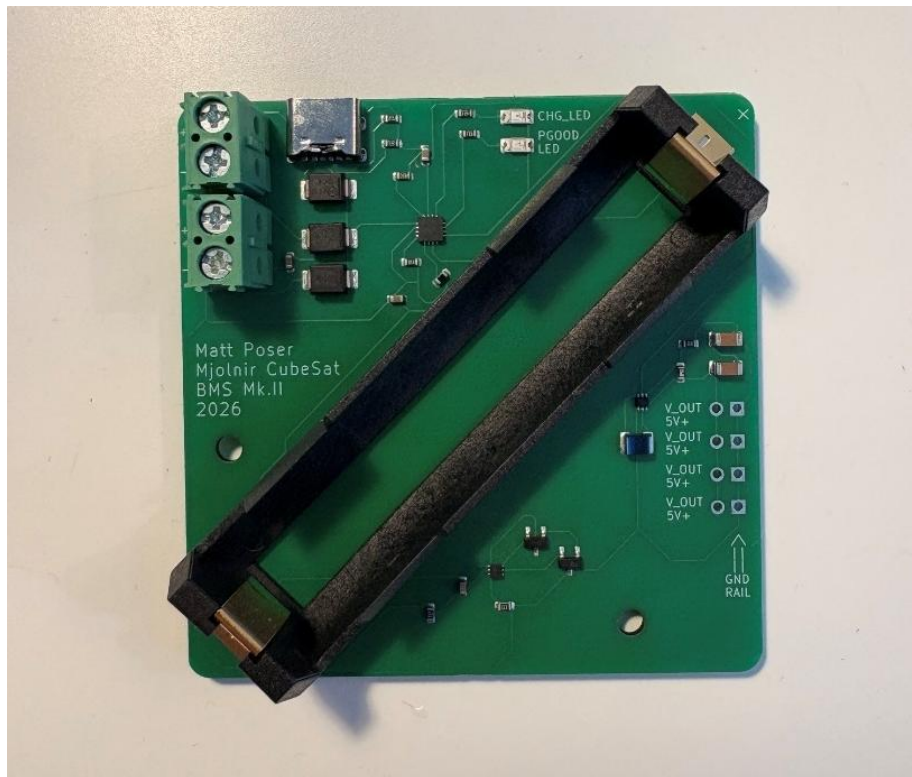


Figure 20- BMS board soldered and assembled from JLCPCB

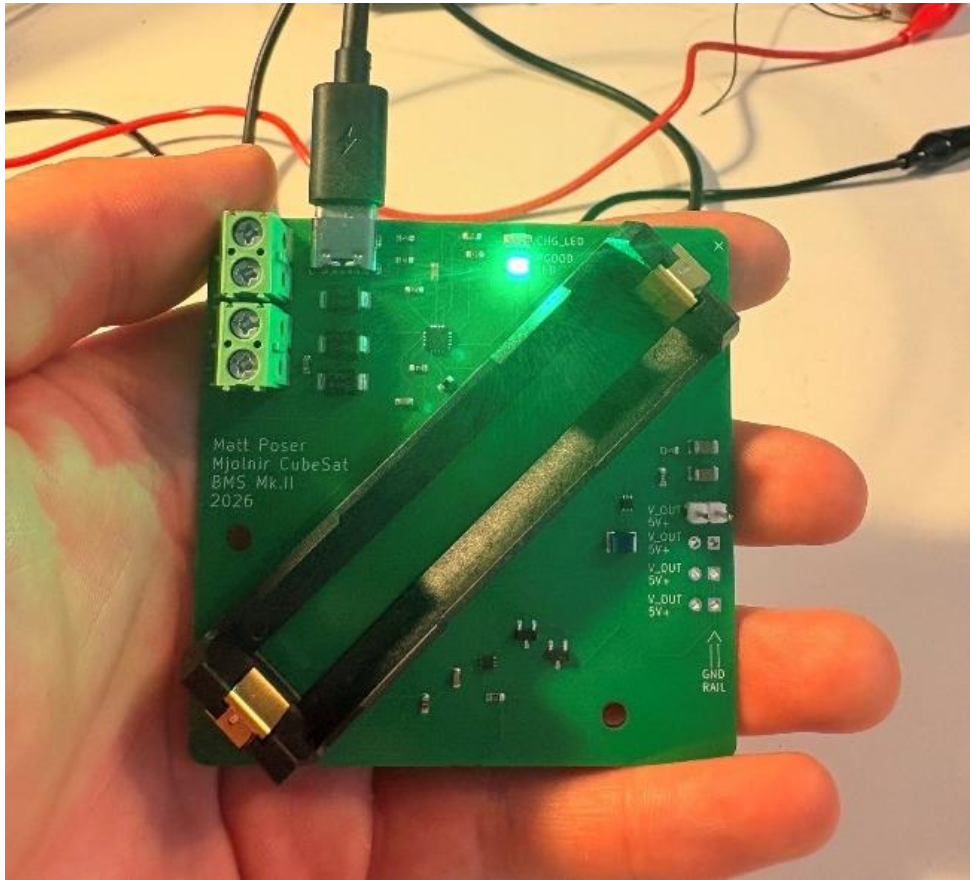


Figure 21- BMS Power test

By far, out of all aspects of this CubeSat project, I struggled the most with the PCB design for the BMS. It was amazing to see it all come together after weeks and weeks of effort, and to finally see the green light turn on during testing was a really awesome sight. This struggling with KiCAD and JLCPCB also prepared me greatly for the much easier reaction wheel motor control PCB design, so much so that I was able to knock out the motor control PCB in one sitting, and have the order placed with JLCPCB within one afternoon.

Tray II – Reaction Wheels

As I discussed in the introductory section of this report, one of the main inspirations for this project was the reaction wheel subsystems seen on satellites and other spacecraft. My 1D approximation of the conservation of angular momentum problem would now be applied along three axes, to control pitch, roll, and yaw of the CubeSat. The first part of this component which needed to be selected was the reaction wheel motors themselves. I was operating under some serious engineering constraints, left with only a 5V power supply and limited space for 3 flywheel motors my options were limited. I settled with a cheap option from amazon, a basic cylindrical 5V DC motor with high enough rpm (roughly 1000) to spin the reaction wheels fast, while maintaining a small footprint within the constrained space of my 1U design.

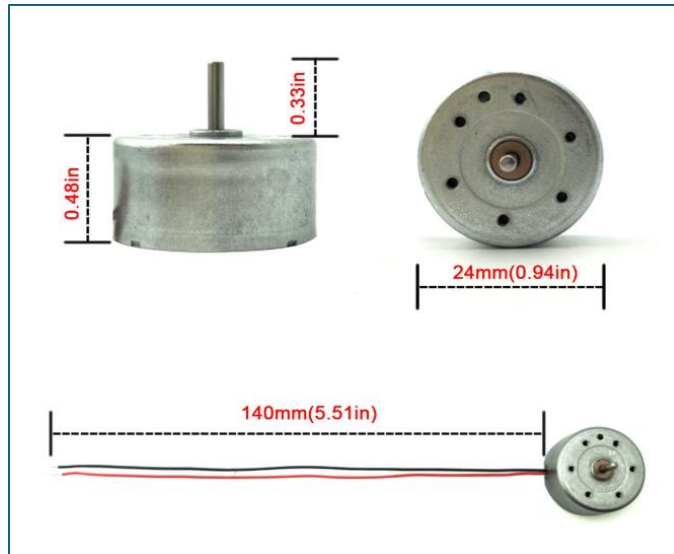


Figure 22- 5V DC motors off amazon

Luckily, I had experience using H-bridge motor control chips before, from my time spent tinkering with an Arduino starter kit in my first year at Queen's. In the starter kit, there was a Texas instruments L293DNE dual H-bridge motor IC, with built in flyback diodes (designed specifically for use with DC motors). This chip operates by using a series of input pins, to control the motor using low-current logic PWM pins (for speed) and low current digital pins (for direction). It has a separate power bus for the motors, so your logic pins are never electrically connected to your motor and thus would not cause damage to the controller (in this case, the logic pins coming directly from the Mjolnir's flight computer). I built a quick test circuit to ensure that the L293DNE was 3.3V logic compatible (Raspberry Pi has 3.3V GPIO I/O & PWM) and began work designing the motor control PCB.

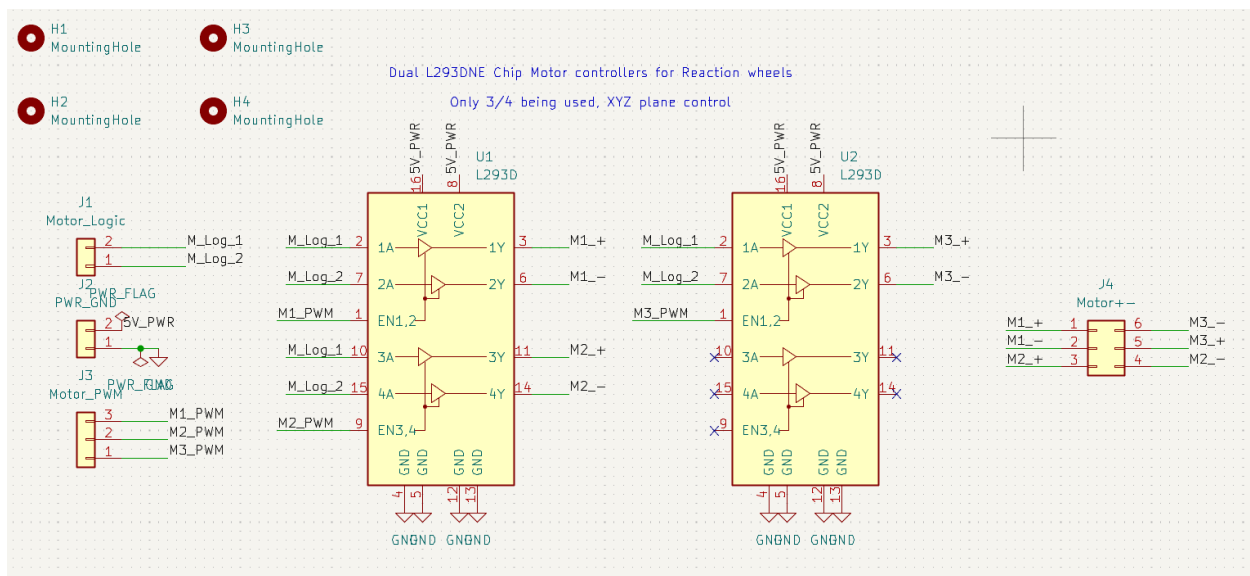


Figure 23- Motor control PCB circuit, using two of the Texas Instruments L293DNE ICs. Two dual H bridges meant 4 total motors could be controlled, but I only required 3 axes of control, thus one was left empty

The above circuit takes in two digital logic pins (0,1 or 1,0) which determines the direction of all motors in the circuit (clockwise versus counterclockwise). The 3 PWM pins on the above circuit receive 3.3V PWM signals of varying duty cycles (set to 490Hz) from the Mjolnir flight computer, and are used to control motor speed on each of the pitch/roll/yaw motors. The choice of 490Hz was made to mirror my past (and well tested) use of these TI chips with an Arduino Uno's PWM outputs, which are set to 490Hz for standard pins. Under normal operation the Mjolnir flight computer would be receiving commands from the groundstation to set the states (direction, speed) of the pitch/roll/yaw motors, which would subsequently be outputted to these pins on the motor control board to interface with the reaction wheel motors. The circuit was traced out for the above PCB, and all the soldering for these boards was completed by me as opposed to the BMS which was assembled by JLCPCB.

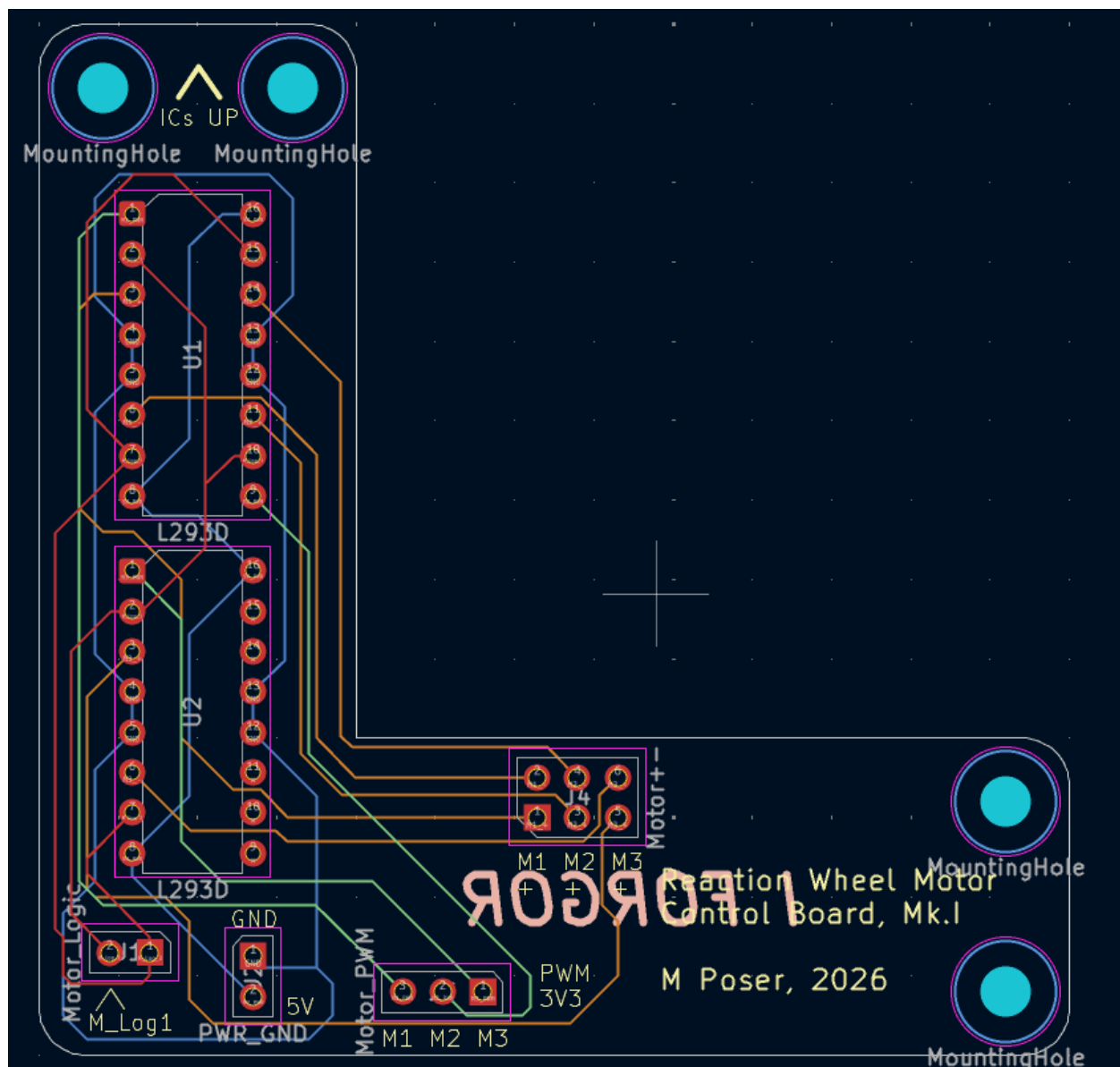


Figure 24- motor control traces. Also experimented with more PCB layers for drawing traces

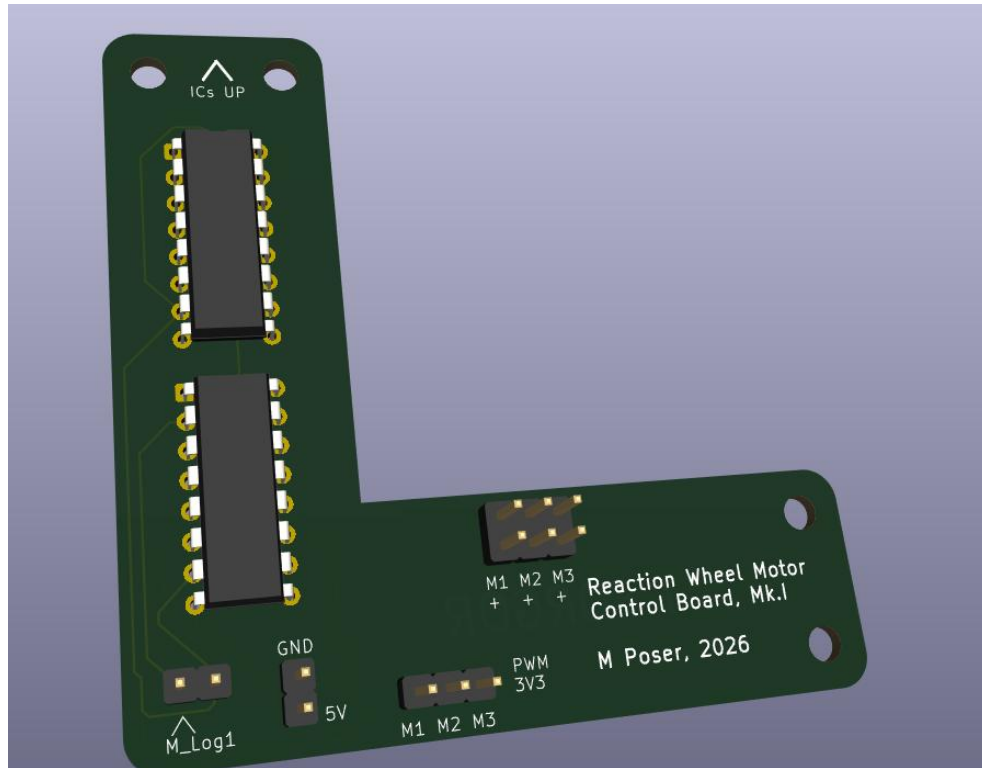


Figure 25- KiCAD 3D render of the motor control PCB

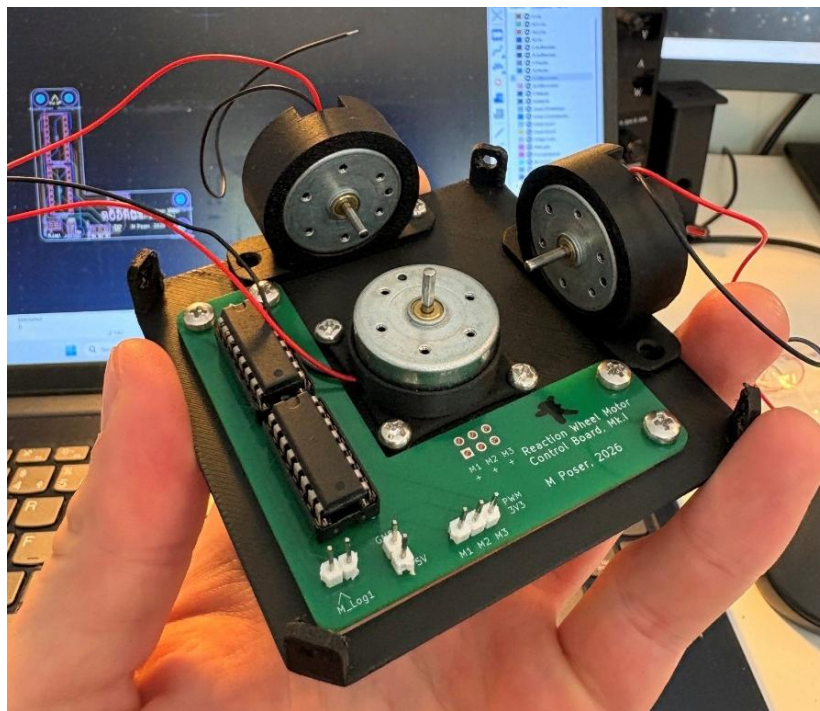


Figure 26- Motor control PCB soldered and fastened to reaction wheel tray along with reaction wheel motors. Flywheels would be printed and assembled later, along with all wiring connections

Contrary to the BMS PCB design, I had a lot of fun making this motor control PCB, and of all the trays on the CubeSat this one is by far the cleanest and the nicest organized. This board underwent basic testing before final soldering and assembly, and performed admirably. The 5V motors also (surprisingly so) pulled much less current than expected at full draw, meaning there was not much strain on the BMS power supply even while running alongside the flight computer & radio module in active operation.

Tray III – Flight Computer, IMU, Radio, Startracker

This final tray on the CubeSat contained much of the finer components in the design, and can be seen as the ‘brain’ of the Mjolnir CubeSat. This final tray would contain the central flight computer, radio module, inertial measurement unit, and (after iteration) the camera module. As well, for the sake of the radio antenna sticking out of the top of the CubeSat, this tray would be mounted at the top. Before completing any major wiring, a basic layout of the top tray was printed and assembled, to assess space constraints.

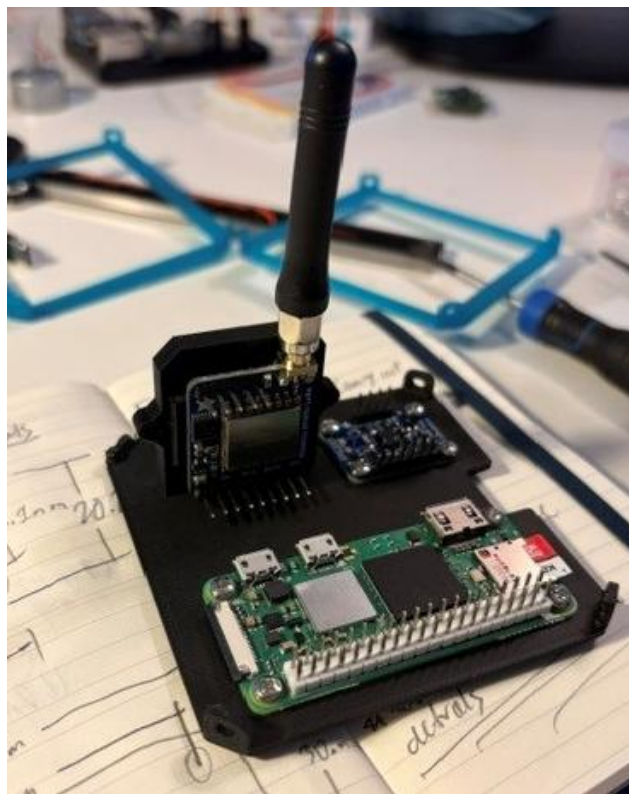


Figure 27- basic layout of Adafruit BNO055 IMU board, Mjolnir flight computer, LoRa radio module



Figure 28- top tray assembled on frame with BMS



Figure 29- Top tray assembled on frame with BMS (2)

Once this basic orientation was figured out, the next component to be added was the PiCamera module for the star tracker. Based on the length of the available camera cable with the module, this also was chosen to be mounted and included in this top tray. Shown below is the 3D model of the top tray, with the outwards facing mount for the PiCamera module. It was noted here that the wall design on this side of the CubeSat would need to have a hole/opening in its face to accommodate the camera's field of view facing outwards.

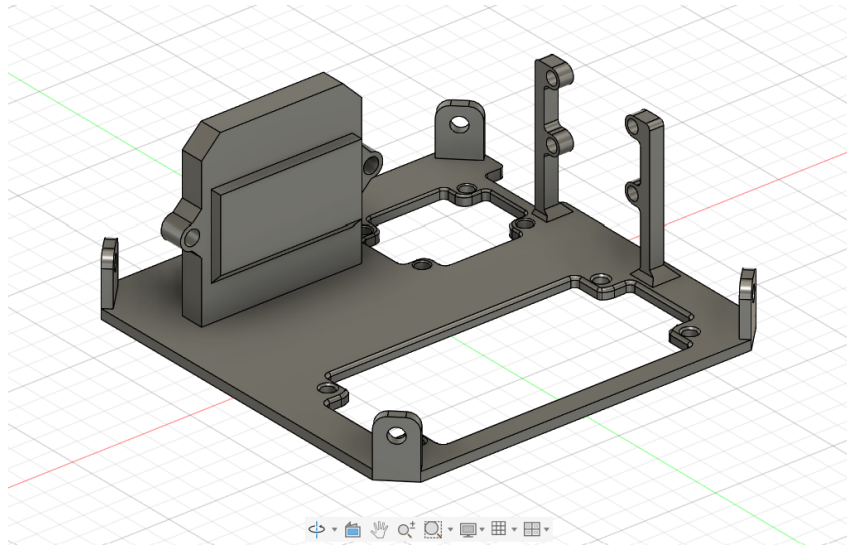


Figure 30- Top tray with PiCamera mounting arms extruded from the base of the tray

It is pertinent to note that this tray was impossibly hard to print, the small thin arm extrusions for the PiCamera were very finicky, and often broke either in printing or in assembly afterwards when fastening the camera on. Eventually, all components were mounted on the tray, and I began initial testing of the flight computer software. Shown below is a photo of the top tray with some of its initial wiring completed (IMU & reaction wheel controls still disconnected from flight computer).

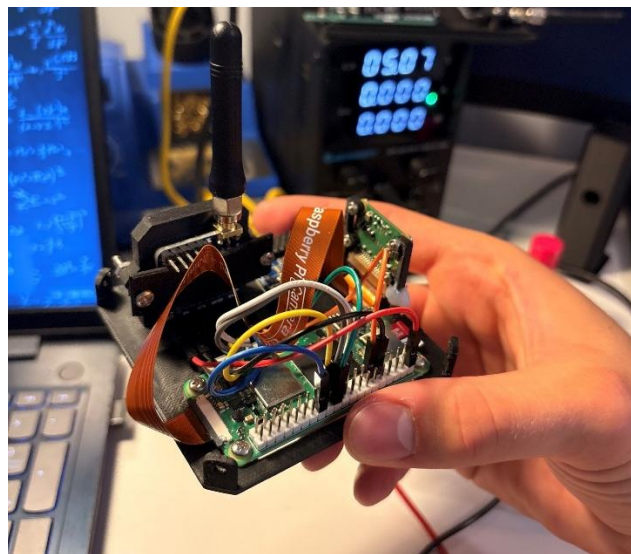


Figure 31- top tray assembled with PiCamera and LoRa radio module

Flight Computer Software

The software on the flight computer was very fun to write, and served as the culmination of the project (alongside the software & hardware of the groundstation, next section). As shown in the above general architecture section, the software on the flight computer is all written in python as a series of sub-scripts which communicate with one another using mqtt architecture (mqtt-paho python library). This mqtt architecture was something I had the opportunity to learn about while on my co-op placement at OPG, and is definitely one of the cooler items in my software 'bag of tricks'. Each script revolves around the same general architecture:

1. Check for a message on an incoming mqtt topic which the script is subscribed to
2. Do something with that packet/message
3. Once the function has been run either publish a message to another topic or report success
4. Continue loop

While unfortunately I do not have the page count in this report to explain the architecture of mqtt at length, the long and short of it is that it is a way for different python scripts running completely separately from one another to exchange information in an asynchronous fashion using built in functions in the paho-mqtt package. The flight computer sets up a local mqtt server/broker, and paho-mqtt allows scripts to publish and read packages from specific 'topics' (like directories) within this local server in a totally asynchronous fashion. What this looks like for the flight computer is that I can have a radio script running which receives IMU (inertial measurement unit) data via a separate script interfacing with my Adafruit IMU and publishing the sensor packets to a specific IMU/PACKET topic. Meanwhile, that same script can publish command data received from its radio to a COMMAND/ topic, which is then received by the command handler script running on a separate python instance. The flow of this controlled madness is outlined in the general architecture section's block diagram, and with some reading of the code on the GitHub can be well understood. I tried to write the flight computer code with minimal usage of AI, and thus in some places it looks clunky. While it can be improved, I am happy because it is original code which runs without error. Shown below is a snippet of code, the IMU controller script which publishes IMU data to a topic read by the main LoRa radio control script.


```

IMU.py > ...
1  #placeholder script for IMU board & data
2
3  #the goal of this script is to broadcast (on regular intervals) sensor packet data
4  #to the IMU/PACKET topic, which the LoRa UDL will read
5
6
7
8  #imports
9  import time
10 import board
11 import busio
12 import adafruit_bno055
13 import paho.mqtt.client as mqtt
14
15
16
17 #mqtt connection stuff here (this script only publishes)
18 def on_connect(client, userdata, flags, reason_code, properties):
19     print(f"mqtt localhost connected with result code {reason_code}")
20
21 def on_publish(client, userdata, mid, reason_codes, properties):
22     print(f"Published message id {mid}")
23
24 IMU_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
25 IMU_client.on_connect = on_connect
26 IMU_client.on_publish = on_publish
27 IMU_client.connect("localhost", 1883)
28 IMU_client.loop_start()
29
30 #the structure of this loop publishing script will be:
31 #on a set timer of 1 second, sleep, then record sensor information,
32 #parse it to correct format, and publish it. Repeat.
33
34 #IMU board setup
35 i2c = busio.I2C(board.SCL, board.SDA) #board pins 5, and 3, respectively
36 sensor = adafruit_bno055.BNO055_I2C(i2c)
37 #use_external_crystal left off (default/internal oscillator) - this board doesn't have one
38 #populated, and forcing it caused persistent None/-128C garbage readings
39

```

```

39
40
41 try:
42     while True:
43         time.sleep(1.5) #maintain the 1Hz floor time limit for ambient temperature readings.
44         #packet timeout speed for the main script will be between 2-5 seconds, so this loop natively
45         #runs faster
46
47         #obtain sensor info
48         CubeSat_angular = sensor.euler #returns a tuple type, as: yaw, roll, pitch
49         CubeSat_temp = sensor.temperature
50
51         #parsing
52         payload_string = f"{CubeSat_angular[2]}, {CubeSat_angular[1]}, {CubeSat_angular[0]}, {CubeSat_temp}C"
53         #this is to put it in the expected pitch, roll, yaw, *C format
54
55
56         #publishing
57         IMU_client.publish("IMU/PACKET", payload_string, retain=True)
58         #this retain=True will allow new subscribers to be fed immediately the newest packet
59
60
61 finally:
62     print("Flight Computer Uplink/Downlink Deactivated")
63     IMU_client.loop_stop()
64     IMU_client.disconnect()
65
66
67

```

Figure 32- IMU sub-script, interfaces with Adafruit IMU to obtain attitude and temperature readings, publishes sensor packet to an mqtt topic read by the main LoRa script, which is then sent down to the groundstation as a sensor packet

Groundstation, Software & Hardware

Unlike the software written for the flight computer, I had a (relative) distaste for frontend design, and largely used AI to ‘vibecode’ this frontend portion of the project. As I mentioned earlier in this report, what I hoped was that the groundstation would be like an operator’s interface; allowing for the fast visualization of the sensor telemetry coming from the CubeSat, as well as providing the operator with a command panel and preset commands to broadcast to the CubeSat to perform onboard functions. The final command panel frontend is shown below, and the feature list discussed afterwards:

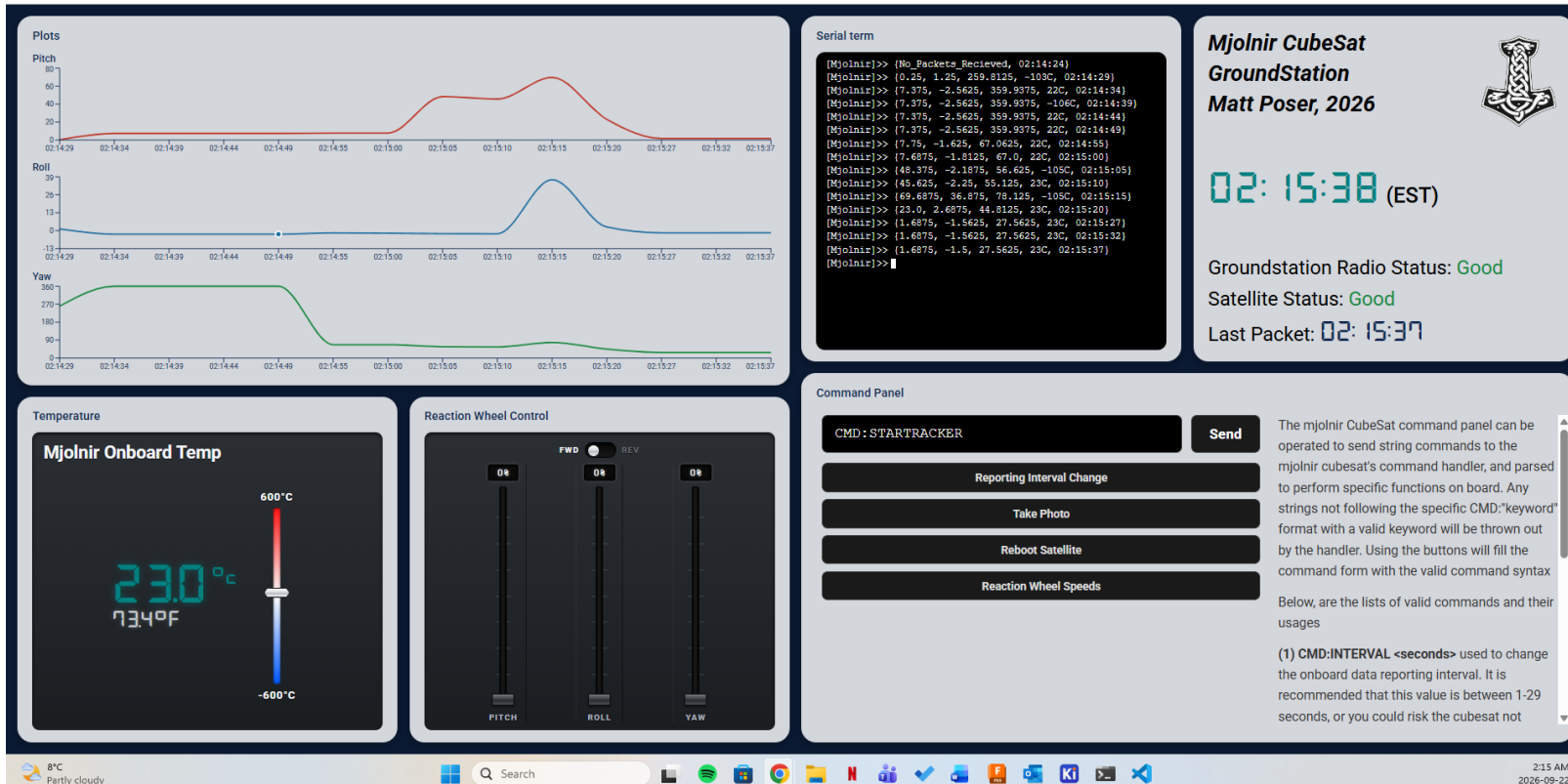


Figure 33- Mjolnir CubeSat groundstation operator panel, designed fully in JavaScript

As mentioned, the feature list is shown below:

1. Live serial terminal, to display packets exactly as they arrive from the CubeSat
2. Live attitude plots, pitch/roll/yaw angle values plotted against timestamp (EST)
3. Clock and status panel, status messages change if backend loses contact with radio, or if the CubeSat has not published a packet to the groundstation in an exceedingly long time (also displays last packet timestamp)
4. Onboard temperature display, temperature included in the CubeSat telemetry packet
5. Reaction wheel control sliders, control the PWM value and direction for the pitch, roll, yaw reaction wheels
6. Command panel, with guides on proper usage (command handler on the CubeSat just throws out bad commands). There are built in commands to:

- a. Change the reporting interval of onboard sensor telemetry packets being sent from the CubeSat
- b. Take a photo using the CubeSat's onboard camera module
- c. Reboot the Mjolnir flight computer
- d. Control/change the reaction wheel speeds (using slider to set values & direction)

This frontend (again, following the general architecture block diagram) sends command strings and receives sensor packets via a python backend (running on my laptop's native Windows OS). This python backend uses fastAPI to interface and exchange these packets with the JavaScript frontend and uses the pyserial module to send/receive these same packets down to the low-level Arduino which is connected to the groundstation radio module. The Arduino code is nothing fancy, but I did take the time to write it without the use of AI. The Arduino/radio essentially waits for packets to arrive from the CubeSat, and if it receives any then it writes that command packet to the serial bus (thus received by the python backend). If the Arduino reads its serial register to be full (python backend has published a command to the serial bus, to be sent to the CubeSat), then the Arduino reads the contents of its serial register into a string object and sends it up to the CubeSat. Shown below is a photo of the Arduino/LoRa radio groundstation module.

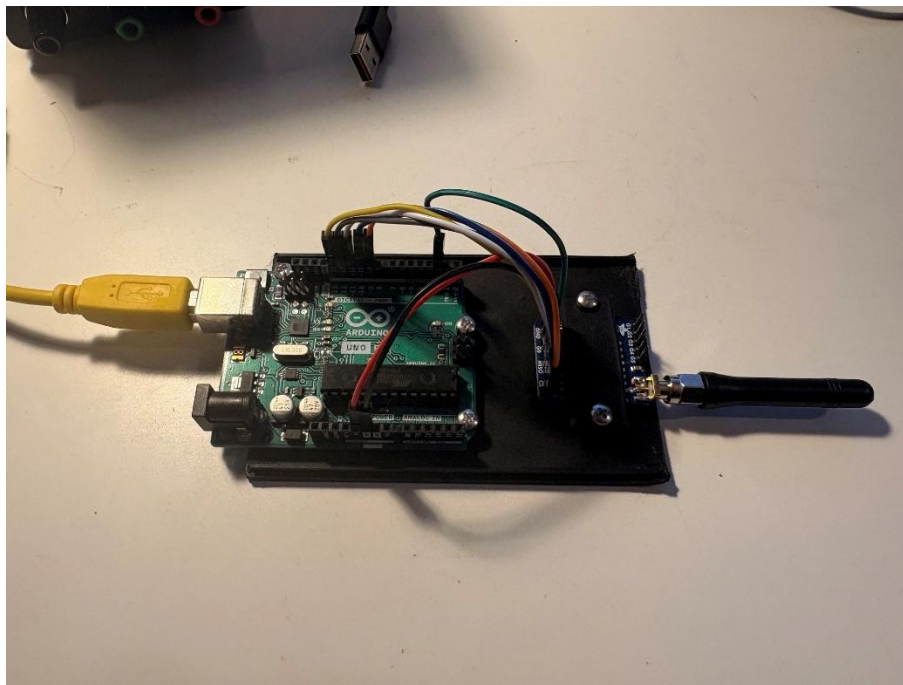


Figure 34- Arduino Uno groundstation module. Yellow serial cable plugs into the USB port of my laptop, to interface with python backend

Final Assembly, Future Project Considerations

With the groundstation completed and all the trays built and tested, it came time to run the entire system in tandem and take readings with the groundstation. Shown below is a photo of the unassembled first electronics test, which involved making all electrical connections between modules and starting up all the onboard sub-scripts.

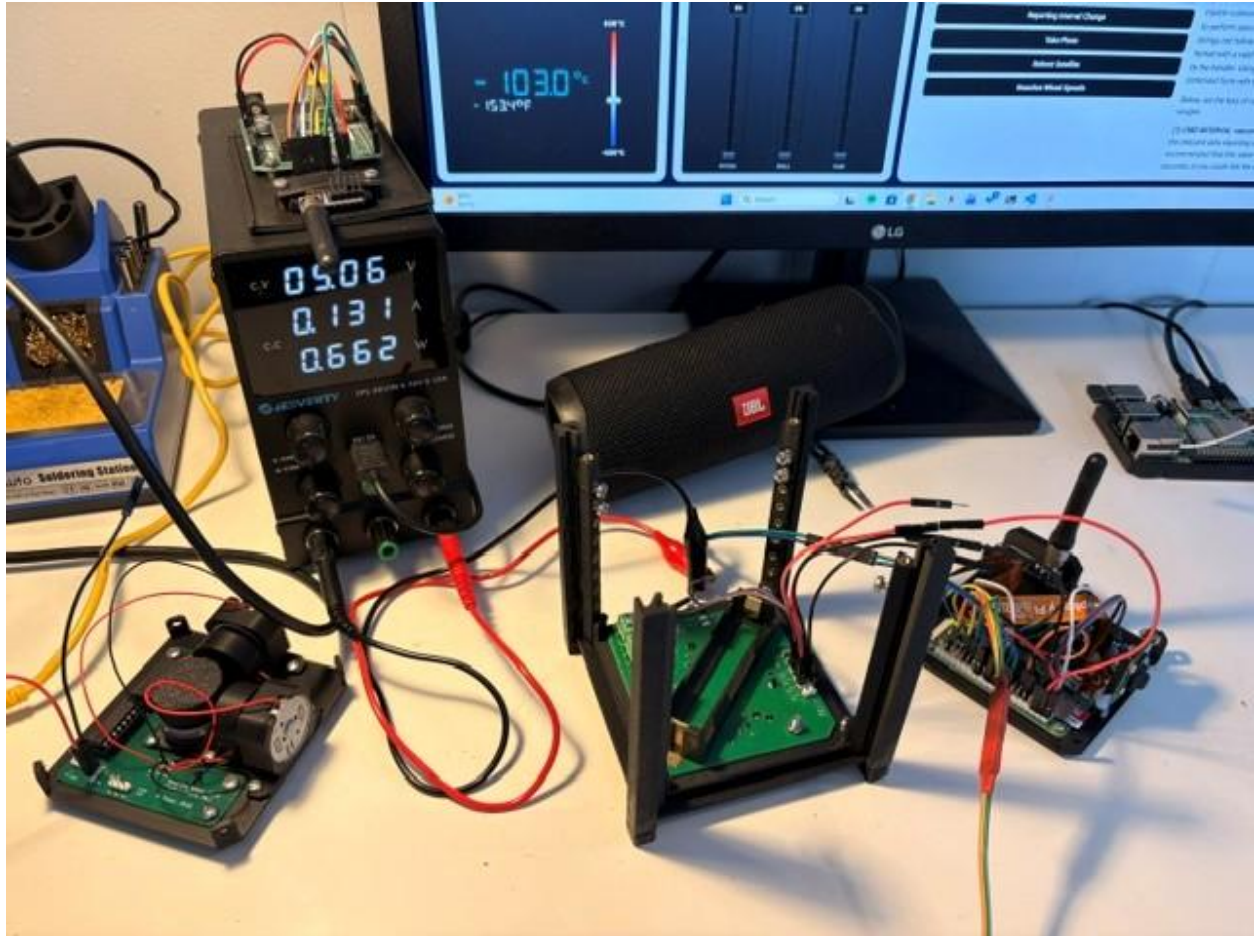
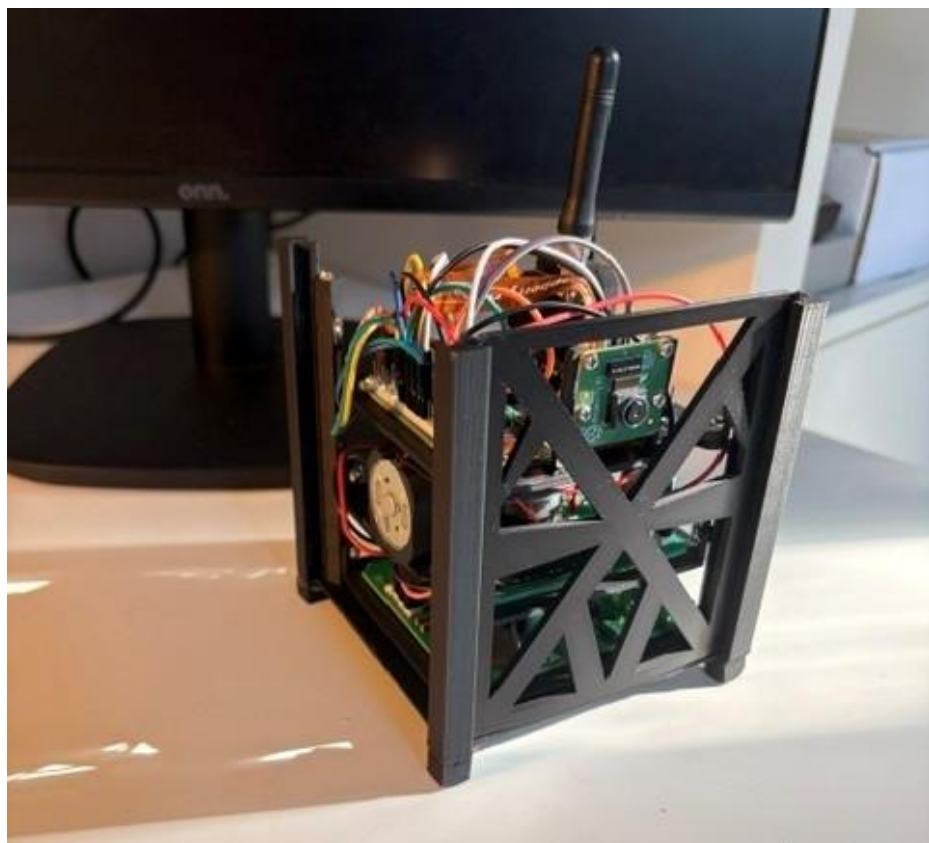
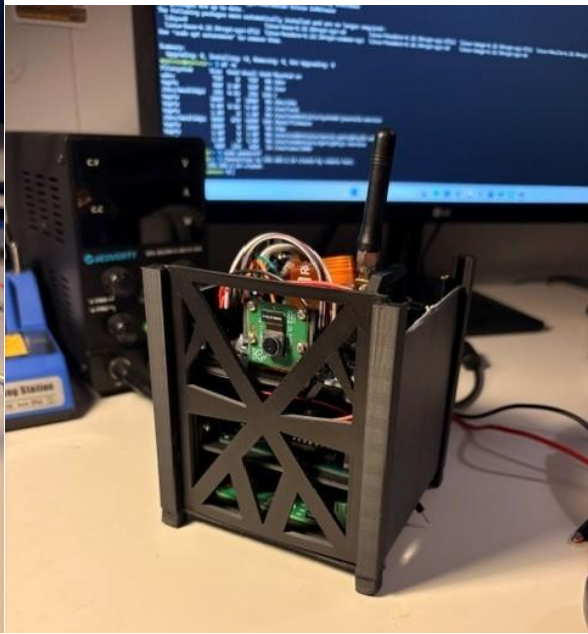
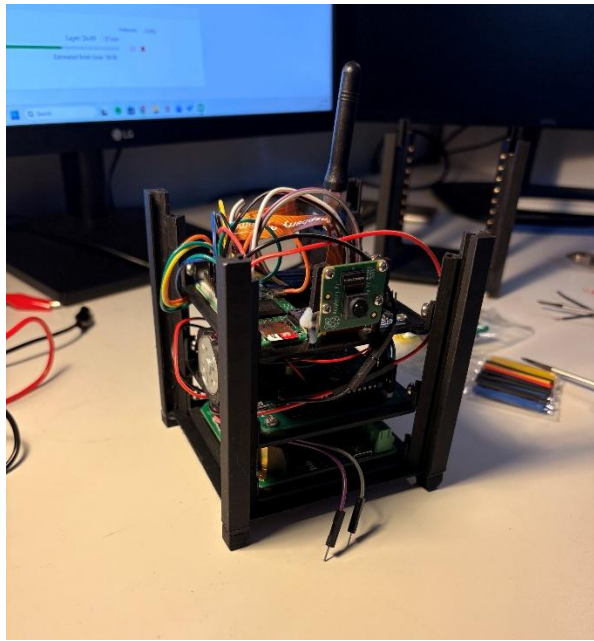


Figure 35- Initial electronics test. See the mess of wires coming out of the flight computer, the 3D printed flywheels on the reaction wheel motor controller, and the bench power supply being used to conserve battery (in lieu of)

This first electronics test ran with surprisingly minimal bugs, and was a large success. This test involved turning on 6/6 sub-scripts (LoRa radio, IMU, command handler, camera, reboot, reaction wheels) and testing all the software, as well as testing the performance of the groundstation sending/receiving commands. For the perceptive reader, you may see that the temperature reading on the groundstation is -103°C . This bug had to do with an erroneous line of code which prompted the IMU to 'use' an external oscillator crystal instead of its own clock, and it did not have an external oscillator crystal soldered onto the breakout board. With that bug fixed, the CubeSat was functioning fully as intended, and I completed the final printing of the walls and the lid for its final assembly. Photos are shown below of intermediate final assembly stages, and then the fully assembled final product.



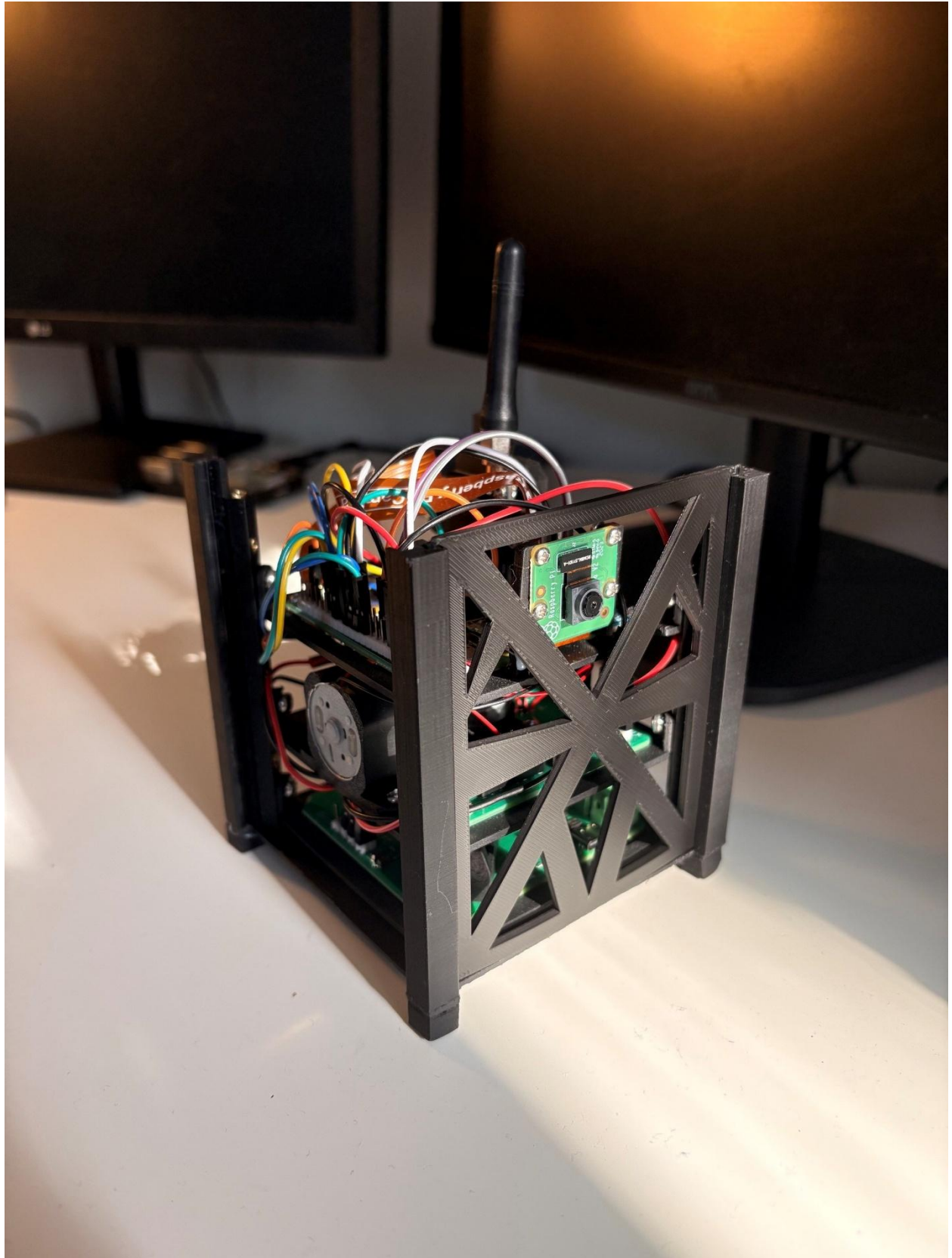






Figure 36- Photos of various stages of final CubeSat assembly

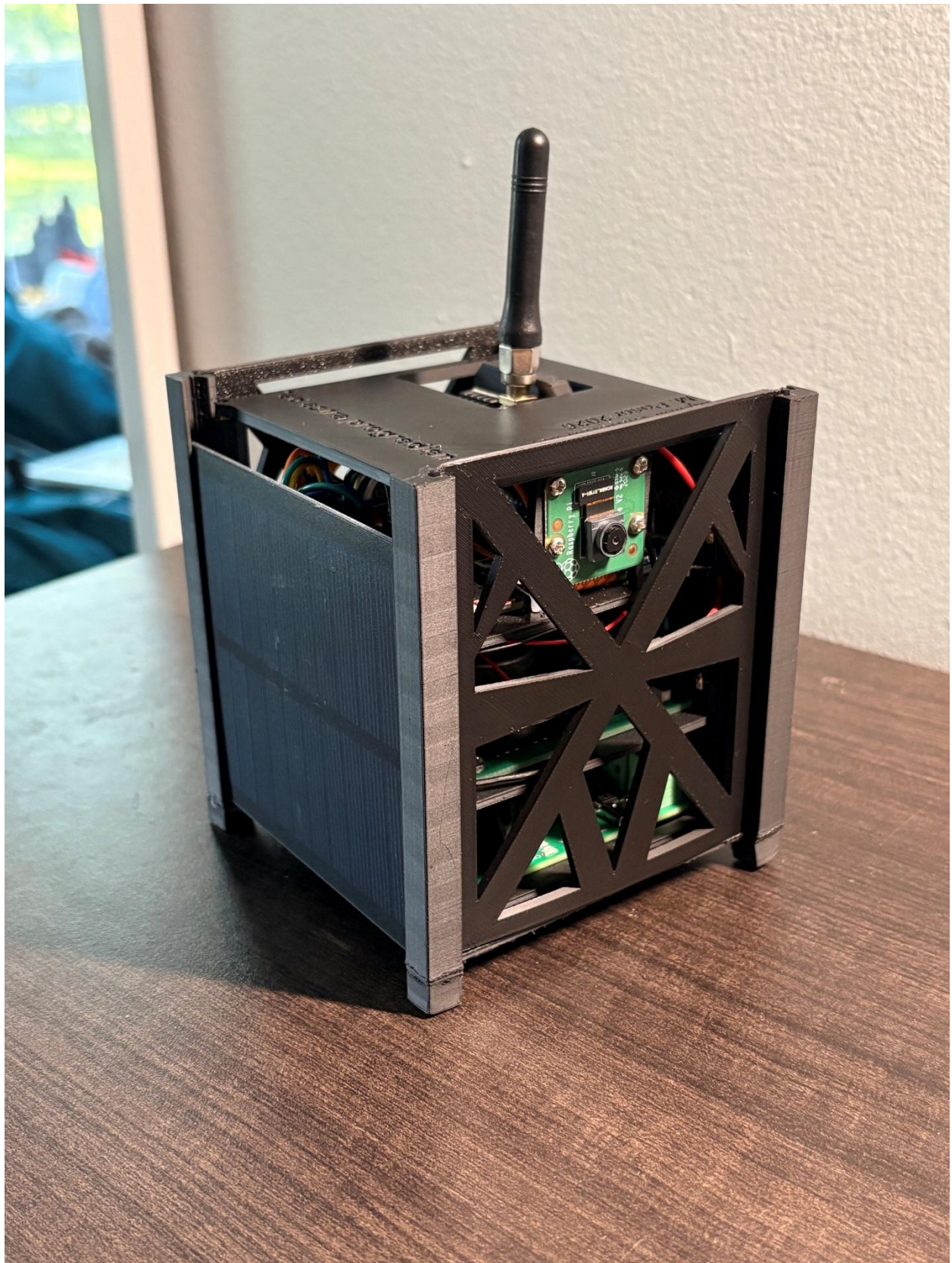


Figure 37- Mjolnir CubeSat, Mk.I

Reflecting on the completion of this project, I definitely was ambitious in my undertaking. I very much underestimated the amount of work and hours which would go into this project, however the constant practice meant that things such as 3D design and PCB design became like second nature. I could easily sit down within the span of an afternoon and knock out a print or a PCB for a specific function after the completion of this project.

Moreso, I am very proud to have completed this project. In terms of feature comparisons and complexity, this project blows my desktop robotic arm out of the water. I faced great challenges over the course of this project and had to think creatively to solve them, which was a nice challenge. I think for my next project (whenever that happens) I'd like to tone down the scope a little bit, to focus on something akin to a singular 'subsystem' or component. I say this because the mechanical design, PCB design, and programming of this project felt like mini projects in and of themselves. I did have a great time learning about all things space and applying some of my knowledge to a potentially industry relevant project, and I accomplished my initial goal to improve my skills in electronic and PCB design.

And, at the end of it all, I may be in the running for the world's most complex desk ornament!